# geometry: the Geometry program, user guide

**geometry-ai999x 1.6.0**, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Geometry covers plane and solid geometry, coordinate geometry, conic sections, polygons and polyhedra, computational geometry (convex hulls, Voronoi, triangulation) and curves and surfaces. This guide is about `geometry`, the model as a self-contained program for Windows and Linux: 78 capabilities in 14 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/geometry-ai999x/); the Python package (`pip install geometry-ai999x`) has its own guide. Every command output printed here is what the program printed.

## Contents


- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability


## 1. Get it

Version 1.6.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

| what | system | download |
|---|---|---|
| Desktop app: Geometry | Windows x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-geometry-1.6.0-windows-x86_64.zip |
| Desktop app: Geometry | Linux x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-geometry-1.6.0-linux-x86_64.tar.gz |
| Desktop app: all models | Windows x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-windows-x86_64.zip |
| Desktop app: all models | Linux x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-linux-x86_64.tar.gz |
| Command line | Windows x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-windows-x86_64.zip |
| Command line | Linux x86-64 | https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-linux-x86_64.tar.gz |

### Desktop app

Geometry in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-geometry-1.6.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-geometry.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-geometry-1.6.0-linux-
x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-geometry-1.6.0-linux-x86_64.tar.gz
./ai999x-geometry-1.6.0-linux-x86_64/ai999x-geometry
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.6.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.6.0-linux-x86_64.tar.gz
./ai999x-1.6.0-linux-x86_64/ai999x
```

### Command line, Linux

The bundle holds one program per model; `geometry` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.6.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.6.0-linux-x86_64:$PATH"' >> ~/.bashrc &&
source ~/.bashrc
geometry --version
```

### Command line, Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.6.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.6.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS" -OutFile
SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.6.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path",
"User") + ";$D", "User")
geometry --version
```

### In your browser

Open https://www.ai999x.com/models/geometry-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

### Python

```
pip install geometry-ai999x
```

## 2. Quick start

### 2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ geometry --version
geometry-ai999x 1.6.0 (Rust build, ax-engine 1.6.0)
```

### 2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ geometry ask "Find the circumcenter and circumradius of the triangle with vertices (0,
0), (4, 0) and (0, 3)."
Read as: Circumcircle: the circle through three points
cx = 2
cy = 1.5
radius = 2.5
  by: Circumcircle: the circle through three points  (intersection of the perpendicular
bisectors, solved in coordinates relative to the first point)
  assumes: exactly 3 points; not collinear
```

### 2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ geometry search "intersection of a circle and a line"
circle_line_intersection            Intersection of a circle and a line  (coordinate
geometry / circles)
line_plane_intersection             Intersection of a line and a plane  (solid coordinate
geometry / lines and planes)
circle_circle_intersection          Intersection of two circles  (coordinate geometry /
circles)
line_intersection                   Intersection point of two lines  (coordinate geometry
/ lines)
segment_intersection                Intersection of two line segments  (computational
geometry / intersections)
angle_between_lines                 Angle between two lines  (coordinate geometry / lines)
convex_polygon_intersection         Area of the intersection of two convex polygons
(computational geometry / polygon clipping)
reflect_point_line                  Reflection of a point in a line  (transformations /
reflections)
… 12 more
```

### 2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ geometry list
angle_between_lines                     Angle between two lines
angle_between_planes                    Angle between two planes
annulus_area                            Area of an annulus (ring)
barycentric_coordinates                 Barycentric coordinates of a point in a triangle
circle_circle_intersection              Intersection of two circles
circle_line_intersection                Intersection of a circle and a line
circle_measures                         Circle: area, circumference and diameter
circular_segment                        Circular segment: area, chord, height, arc
… 70 more
```

### 2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ geometry info circle_line_intersection
Intersection of a circle and a line
  foot of the perpendicular from the centre, then the half-chord sqrt(r^2 - d^2) along the
line
  in   cx       x of the centre [1]
  in   cy       y of the centre [1]
  in   r        radius [1]
  in   a        coefficient of x of the line [1]
  in   b        coefficient of y of the line [1]
  in   c        constant of the line [1]
  out  count    number of intersection points (0, 1 or 2) [1]
  out  x1       x of the first point (ordered by x, then y) [1]
  out  y1       y of the first point [1]
  out  x2       x of the second point [1]
  out  y2       y of the second point [1]
```

### 2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ geometry solve circle_line_intersection cx=0.0 cy=0.0 r=5.0 a=-1.0 b=1.0 c=-1.0
count = 2
x1 = -4
y1 = -3
x2 = 3
y2 = 4
  by: Intersection of a circle and a line  (foot of the perpendicular from the centre, then
the half-chord sqrt(r^2 - d^2) along the line)
  cx = 0
  cy = 0
  r = 5
  a = -1
  b = 1
  c = -1
  assumes: r > 0; a and b not both 0; tangency is decided exactly on the written values;
points ordered by x, then y; absent points are empty
```

### 2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ geometry solve circle_line_intersection cx=0.0 cy=0.0 r=5.0 a=-1.0 b=1.0 c=-1.0 --json
{"outcome":"SOLVED","message":"","value":2.0,"unit":"","solved_for":"count","si":
[["cx",0.0],["cy",0.0],["r",5.0],["a",-1.0],["b",1.0],["c",-1.0]],"values":[["count",2,""],
["x1",-4.0,""],["y1",-3.0,""],["x2",3.0,""],
["y2",4.0,""]],"target":"count","phrase":"number of intersection points (0, 1 or
2)","assumptions":["r > 0","a and b not both 0","tangency is decided exactly on the written
values","points ordered by x, then y; absent points are empty"],"name":"Intersection of a
circle and a line","equation":"foot of the perpendicular from the centre, then the half-
chord sqrt(r^2 - d^2) along the line"}
```

### 2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ geometry solve circle_line_intersection cx=0.0 cy=0.0 r=5.0 a=-1.0 b=1.0
NEEDS_INPUT: circle_line_intersection needs c (constant of the line). Example:
solve('circle_line_intersection', cx=0, cy=0, r=5, a=1, b=-1, c=1).
```

### 2.9 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | geometry batch

`cases.jsonl`:

```
{"key": "circle_line_intersection", "inputs": {"cx": 0.0, "cy": 0.0, "r": 5.0, "a": -1.0,
"b": 1.0, "c": -1.0}}
{"key": "triangle_circumcircle", "inputs": {"points": "[0, 0, 4, 0, 0, 3]"}}
```

```
$ geometry batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":2.0,"unit":"","solved_for":"count","si":
[["cx",0.0],["cy",0.0],["r",5.0],["a",-1.0],["b",1.0],["c",-1.0]],"values":[["count",2,""],
["x1",-4.0,""],["y1",-3.0,""],["x2",3.0,""],
["y2",4.0,""]],"target":"count","phrase":"number of intersection points (0, 1 or
2)","assumptions":["r > 0","a and b not both 0","tangency is decided exactly on the written
values","points ordered by x, then y; absent points are empty"],"name":"Intersection of a
circle and a line","equation":"foot of the perpendicular from the centre, then the half-
chord sqrt(r^2 - d^2) along the line"}
{"outcome":"SOLVED","message":"","value":2.5,"unit":"","solved_for":"radius","values":
[["cx",2.0,""],["cy",1.5,""],["radius",2.5,""]],"target":"radius","phrase":"x of the
circumcentre","assumptions":["exactly 3 points","not collinear"],"name":"Circumcircle: the
circle through three points","equation":"intersection of the perpendicular bisectors,
solved in coordinates relative to the first point"}
```

## 3. Writing inputs

Inputs are written `name=value`, one per argument; `geometry info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

| kind | example | how it is read |
|---|---|---|
| a number with a unit | `"R=0.128 nm"` | any unit of the same kind; converted to SI |
| a number in SI | `R=1.28e-10` | no unit means the SI unit shown by info |
| a choice | `structure=FCC` | one of the values info lists |
| a list or vector | `"v=[1, 2, 3]" or "v=1, 2, 3"` | brackets keep it a list |
| a matrix | `"A=[[1, 2], [3, 4]]"` | one list per row |
| true / false | `inclusive=true` | for yes/no options |

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

## 4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

| outcome | meaning | exit status |
|---|---|---|
| SOLVED | the answer: every output with its unit, the method, the inputs in SI, the conditions assumed | 0 |
| NEEDS_INPUT | an input is missing; the message names it and shows a complete call | 1 |
| INVALID_INPUT | a value or unit of the wrong kind, an unknown input name, or a value outside the conditions | 1 |
| AMBIGUOUS | the inputs allow two different answers; say which one is meant | 1 |
| OUT_OF_SCOPE | the model has no capability for this, or the result would not be meaningful | 1 |

| outcome | meaning | exit status |
|---|---|---|
| (usage) | a command the program does not know: it prints its usage | 2 |

## 5. Scripts and agents: batch and JSON

`geometry solve ... --json` prints one answer as one line of JSON. `geometry batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | geometry batch`. `geometry catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

| answer field | meaning |
|---|---|
| outcome | one of the outcomes above |
| message | why there is no answer (empty when SOLVED) |
| value, unit | the main result in SI units |
| values | every output: [symbol, value, unit] (procedures) |
| target | the quantity solved for |
| si | each numeric input converted to SI: [symbol, value] |
| assumptions | the conditions the answer relies on |
| name, equation | the capability and its equation or method |

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["geometry", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

## 6. Figures

`geometry figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

## 7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `geometry_ai999x.wasm` (https://www.ai999x.com/compute/rust/geometry_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("geometry_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

## 8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `geometry info <id>` shows the same inputs and outputs in the terminal.

### Computational geometry (12)

#### Area of the intersection of two convex polygons, `convex_polygon_intersection`

The area common to two convex polygons given by their vertices.

$$A(P \cap Q)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `p` | in | first convex polygon's vertices in order | , |
| `q` | in | second convex polygon's vertices in order | , |
| `area` | out | area of the intersection (0 if they do not overlap) | , |

**Applies when:** both polygons simple and convex.

```
$ geometry solve convex_polygon_intersection "p=[0, 0, 2, 0, 2, 2, 0, 2]" "q=[1, 1, 3, 1,
3, 3, 1, 3]"
area = 1
  by: Area of the intersection of two convex polygons  (Sutherland-Hodgman clipping of one
polygon by the other's edges, then the shoelace area)
  assumes: both polygons simple and convex
```

**What the worked example means:** This calculation determines the common region shared by two convex polygons, defined by their ordered vertices. Clipping one polygon against the edges of the other and applying the shoelace formula to the resulting shape gives the precise overlap area. The result, 1, represents the total size of this intersection in square units, indicating how much space the two shapes occupy simultaneously.

#### Closest pair of points, `closest_pair`

The smallest distance between any two of a set of points in the plane.

$$\delta = \min_{i<j} \|p_i - p_j\|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points: x1 y1, x2 y2, ... | , |
| `distance` | out | smallest pairwise distance | , |

**Applies when:** at least 2 points, at most 2000; a repeated point gives distance 0.

```
$ geometry solve closest_pair "points=[0, 0, 5, 5, 1, 1, 9, 2, 6, 6.5]"
distance = 1.414213562
  by: Closest pair of points  (sort by x and sweep, stopping each scan once the x-gap
exceeds the best distance)
  assumes: at least 2 points, at most 2000; a repeated point gives distance 0
```

**What the worked example means:** This calculation determines the smallest Euclidean distance between any two points in the provided set. By sorting the points and sweeping through them, the algorithm identifies the minimum separation, which is reported as 1.41421. This value represents the tightest proximity found among all possible pairs.

### Convex hull of points in the plane, `convex_hull`

The convex hull of a set of points: its vertices (counterclockwise), their number, its area and perimeter.

$$\mathrm{conv}(S) = \{\sum \lambda_i p_i : \lambda_i \geq 0, \sum \lambda_i = 1\}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points: x1 y1, x2 y2, ... | , |
| `hull` | out | hull vertices [x, y], counterclockwise from the smallest x (then smallest y) | , |
| `n_vertices` | out | number of hull vertices | , |
| `area` | out | area of the hull | , |
| `perimeter` | out | perimeter of the hull | , |

**Applies when:** at least 3 points, not all collinear, at most 5000; points on a hull edge between two vertices are not vertices; repeated points count once.

```
$ geometry solve convex_hull "points=[0, 0, 2, 1, 4, 0, 3, 3, 1, 2, 2, 4]"
hull = [[0.0,0.0],[4.0,0.0],[3.0,3.0],[2.0,4.0]]
n_vertices = 4
area = 9
perimeter = 13.04862718
  by: Convex hull of points in the plane  (Andrew's monotone chain with exact (rational)
orientation tests; shoelace area)
  assumes: at least 3 points, not all collinear, at most 5000; points on a hull edge
between two vertices are not vertices; repeated points count once
```

**What the worked example means:** This calculation determines the smallest convex polygon that contains all provided points. The result identifies 4 specific vertices arranged in counterclockwise order, starting from the point with the smallest x-coordinate. The enclosed region has an area of 9 and a boundary length of 13.0486. This process assumes the input set contains at least three non-collinear points to form a valid two-dimensional shape.

### Delaunay triangulation and Voronoi diagram: counts, `delaunay_voronoi_counts`

Numbers of Delaunay triangles and edges, hull vertices, Voronoi vertices and unbounded Voronoi cells of points in general position.

$$T = 2n - 2 - h, \quad E = 3n - 3 - h$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points: x1 y1, x2 y2, ... (3 to 25) | , |
| `triangles` | out | Delaunay triangles 2n - 2 - h | , |
| `edges` | out | Delaunay edges 3n - 3 - h | , |
| `hull_vertices` | out | h, vertices of the convex hull | , |
| `voronoi_vertices` | out | Voronoi vertices (one per triangle) | , |
| `unbounded_cells` | out | unbounded Voronoi cells (one per hull vertex) | , |

**Applies when:** 3 to 25 distinct points; general position: no three collinear, no four concyclic (checked exactly).

```
$ geometry solve delaunay_voronoi_counts "points=[0, 0, 4, 0, 0, 3, 5, 4, 2, 1]"
triangles = 4
edges = 8
hull_vertices = 4
voronoi_vertices = 4
unbounded_cells = 4
  by: Delaunay triangulation and Voronoi diagram: counts  (exact general-position checks,
the convex hull, then Euler's formula for a triangulation; Voronoi is the dual)
  assumes: 3 to 25 distinct points; general position: no three collinear, no four concyclic
(checked exactly)
```

**What the worked example means:** Given a set of distinct points in general position, this calculation determines the structural counts for both the Delaunay triangulation and its dual, the Voronoi diagram. The number of triangles is derived from the total point count minus the convex hull vertices, resulting in 4 triangles. Correspondingly, there are 8 edges connecting these points. The Voronoi diagram mirrors this structure with 4 vertices, matching the triangle count, and 4 unbounded regions, which correspond to the 4 vertices on the outer boundary of the point cloud.

### Distance from a point to a segment, `point_segment_distance`

The shortest distance from a point to the segment AB and the closest point of the segment.

$$t^* = \mathrm{clamp}\left(\frac{(P - A)\cdot(B - A)}{\|B - A\|^2}, 0, 1\right), \quad d = \|P - (A + t^*(B - A))\|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `x1` | in | x of A | , |
| `y1` | in | y of A | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `x2` | in | x of B | , |
| `y2` | in | y of B | , |
| `distance` | out | shortest distance | , |
| `qx` | out | x of the closest point | , |
| `qy` | out | y of the closest point | , |

**Applies when:** A != B.

```
$ geometry solve point_segment_distance px=3 py=4 x1=0 y1=0 x2=6 y2=0
distance = 4
qx = 3
qy = 0
  by: Distance from a point to a segment  (project P onto AB, clamp the parameter to [0,
1])
  px = 3
  py = 4
  x1 = 0
  y1 = 0
  x2 = 6
  y2 = 0
  assumes: A != B
```

**What the worked example means:** This calculation determines the minimum Euclidean distance between a specific point and a finite line segment defined by two distinct endpoints. By projecting the point onto the infinite line containing the segment, the algorithm identifies the parameter that minimizes the squared distance, then clamps this value to ensure the resulting point lies within the segment bounds. The final output provides both the scalar distance 4 and the coordinates of the nearest point on the segment, (3, 0), which represents either an interior projection or one of the endpoints depending on the geometry.

**Farthest pair of points (diameter of a point set), `farthest_pair`**

The largest distance between any two of a set of points in the plane.

$$D = \max_{i<j} \|p_i - p_j\|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points: x1 y1, x2 y2, ... | , |
| `distance` | out | largest pairwise distance | , |

**Applies when:** at least 2 points, at most 5000.

```
$ geometry solve farthest_pair "points=[0, 0, 3, 4, 1, 1, 2, 2]"
distance = 5
  by: Farthest pair of points (diameter of a point set)  (the farthest pair are vertices of
the convex hull: hull first (monotone chain), then all hull-vertex pairs)
  assumes: at least 2 points, at most 5000
```

**What the worked example means:** This calculation determines the largest distance between any two points in the provided set, known as the diameter. The method first identifies the convex hull of the points and then computes the distances between all pairs of vertices on this hull to find the maximum value. The resulting diameter is 5, representing the greatest separation found among the input coordinates.

### Hausdorff distance between two point sets, `hausdorff_distance`

The Hausdorff distance between two finite point sets and both directed distances.

$$d_H(A,B) = \max\{\sup_a \inf_b \|a-b\|,\ \sup_b \inf_a \|a-b\|\}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | first set: x1 y1, x2 y2, ... | , |
| `b` | in | second set | , |
| `distance` | out | max of the two directed distances | , |
| `from_a` | out | max over A of the distance to the nearest point of B | , |
| `from_b` | out | max over B of the distance to the nearest point of A | , |

**Applies when:** 1 to 1000 points per set.

```
$ geometry solve hausdorff_distance "a=[0, 0, 1, 0]" "b=[0, 0, 3, 4]"
distance = 4.472135955
from_a = 1
from_b = 4.472135955
  by: Hausdorff distance between two point sets  (directed distances by exhaustive nearest-point search)
  assumes: 1 to 1000 points per set
```

**What the worked example means:** This calculation determines the Hausdorff distance between two point sets by evaluating the maximum nearest-neighbor distance in both directions. The result, 4.47214, represents the greatest separation found when moving from any point in the first set to its closest counterpart in the second set, or vice versa. The individual directional components are reported as 1 for points originating in the first set and 4.47214 for those originating in the second set, ensuring that the final metric captures the worst-case deviation between the two groups.

### Intersection of two line segments, `segment_intersection`

Whether segments AB and CD meet, and where: at one point, along an overlap, or not at all.

$$A + t(B-A) = C + u(D-C), \quad 0 \le t,u \le 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the endpoints A, B, C, D: xA yA, xB yB, xC yC, xD yD | , |
| `status` | out | point / overlap / none | , |
| `x` | out | x of the common point (point only) | , |
| `y` | out | y of the common point (point only) | , |

**Applies when:** each segment has distinct endpoints; decided exactly on the written coordinates (touching counts as meeting).

```
$ geometry solve segment_intersection "points=[0, 0, 4, 4, 0, 4, 4, 0]"
status = point
x = 2
y = 2
  by: Intersection of two line segments  (exact orientation tests, then the intersection
parameter in rational arithmetic; collinear overlaps by interval)
  assumes: each segment has distinct endpoints; decided exactly on the written coordinates
(touching counts as meeting)
```

**What the worked example means:** This calculation determines the geometric relationship between two line segments defined by their endpoints. By applying exact orientation tests to the coordinates of points A, B, C, and D, the algorithm checks if the segments cross or touch. The result indicates that the segments share a common point, with the intersection occurring specifically at x-coordinate 2 and y-coordinate 2. This precise location is derived from solving the parametric equations of both lines under rational arithmetic, ensuring that boundary cases where segments merely touch are correctly identified as intersections.

**Minimum-area enclosing rectangle, `min_area_rectangle`**

The area (and sides) of the smallest-area rectangle, in any orientation, enclosing the points.

$$A^* = \min_{e \in \text{hull edges}} w_e \, h_e$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points: x1 y1, x2 y2, ... | , |
| `area` | out | minimum area | , |
| `width` | out | shorter side of that rectangle | , |
| `length` | out | longer side | , |

**Applies when:** at least 3 points, not all collinear; when two orientations tie in area, the sides of the first found are reported.

```
$ geometry solve min_area_rectangle "points=[0, 0, 2, 2, 4, 0, 2, -2]"
area = 8
width = 2.828427125
length = 2.828427125
  by: Minimum-area enclosing rectangle  (convex hull, then one side of the optimal
rectangle lies along a hull edge: try every edge (rotating calipers))
  assumes: at least 3 points, not all collinear; when two orientations tie in area, the
sides of the first found are reported
```

**What the worked example means:** This calculation determines the smallest possible rectangle that contains all provided points, regardless of rotation. By examining every edge of the convex hull, the algorithm identifies the orientation where the product of the width and height is minimized. The result reports this minimal area as 8, alongside the specific dimensions of the shorter side 2.82843 and the longer side 2.82843. This approach ensures a tight fit for spatial analysis or object bounding.

### Nearest neighbour of a query point, `nearest_neighbor`

Which of a list of points is nearest to a query point, and how far it is.

$$k^* = \arg\min_k \|p_k - q\|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the query point | , |
| `py` | in | y of the query point | , |
| `points` | in | the candidate points: x1 y1, x2 y2, ... | , |
| `index` | out | position of the nearest point in the list (1 = first) | , |
| `distance` | out | its distance | , |
| `nx` | out | its x | , |
| `ny` | out | its y | , |

**Applies when:** ties go to the earliest point in the list.

```
$ geometry solve nearest_neighbor px=3 py=3 "points=[0, 0, 5, 5, 2, 3]"
index = 3
distance = 1
nx = 2
ny = 3
  by: Nearest neighbour of a query point  (exhaustive search on squared distances)
  px = 3
  py = 3
  assumes: ties go to the earliest point in the list
```

**What the worked example means:** This calculation determines which candidate point from the provided list is closest to the specific query location defined by coordinates 3 and 3. By evaluating the Euclidean distance between the query point and every candidate, the algorithm identifies the minimum distance. The result indicates that the point at index 3 in the original list is the nearest neighbor, situated at coordinates 2 and 3. The computed distance between the query and this nearest point is 1. If multiple points were equidistant, the one appearing earliest in the input list would be selected.

### Point in polygon test, `point_in_polygon`

Whether a point lies inside, outside or on the boundary of a simple polygon.

$$\text{inside} \iff \#\{\text{edges crossed by the ray } y = p_y, x > p_x\} \text{ is odd}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `points` | in | polygon vertices in order: x1 y1, x2 y2, ... | , |
| `location` | out | inside / outside / boundary | , |

**Applies when:** simple polygon with at least 3 vertices; decided exactly on the written coordinates.

```
$ geometry solve point_in_polygon px=2 py=3 "points=[0, 0, 5, 0, 5, 5, 0, 5]"
location = inside
  by: Point in polygon test  (exact boundary test, then the even-odd ray-crossing rule with
exact rational arithmetic)
  px = 2
  py = 3
  assumes: simple polygon with at least 3 vertices; decided exactly on the written
coordinates
```

**What the worked example means:** This calculation determines the spatial relationship between a specific point and a simple polygon. Using exact rational arithmetic, it first checks if the point lies on any edge of the polygon defined by [0, 0, 5, 0, 5, 5, 0, 5]. If not on the boundary, it applies the even-odd ray-crossing rule by casting a horizontal ray from the point's x-coordinate 2 and y-coordinate 3 towards positive infinity. The final result, inside, indicates whether the number of intersected edges is odd (inside) or even (outside), ensuring precision without floating-point errors.

#### Smallest enclosing circle of points, `min_enclosing_circle`

Centre and radius of the smallest circle that contains every given point.

$$\min_{c,r} r \;\text{ s.t. } \|p_i - c\| \le r \;\forall i$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points: x1 y1, x2 y2, ... | , |
| `cx` | out | x of the centre | , |
| `cy` | out | y of the centre | , |
| `radius` | out | radius | , |

**Applies when:** 1 to 2000 points; the smallest enclosing circle is unique.

```
$ geometry solve min_enclosing_circle "points=[0, 0, 4, 0, 2, 1]"
cx = 2
cy = 0
radius = 2
  by: Smallest enclosing circle of points  (Welzl's incremental algorithm (fixed shuffled
order), circles through 2 or 3 support points)
  assumes: 1 to 2000 points; the smallest enclosing circle is unique
```

**What the worked example means:** This calculation determines the unique smallest circle that contains all provided points. The resulting centre coordinates are given as 2 for the horizontal position and 0 for the vertical position, while the size is defined by the radius 2. This represents the tightest possible boundary around the data set, ensuring every input point lies within or on the edge of this circle.

### Conic sections (2)

#### Classify a conic section from its general equation, `conic_classify`

Whether A x^2 + B xy + C y^2 + D x + E y + F = 0 is a circle, ellipse, parabola, hyperbola, a degenerate conic or has no real points.

$$\Delta = \det \begin{pmatrix} A & B/2 & D/2 \\ B/2 & C & E/2 \\ D/2 & E/2 & F \end{pmatrix}, \quad B^2 - 4AC \lessgtr 0$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `A` | in | coefficient of x^2 | , |
| `B` | in | coefficient of xy | , |
| `C` | in | coefficient of y^2 | , |
| `D` | in | coefficient of x | , |
| `E` | in | coefficient of y | , |
| `F` | in | constant term | , |
| `type` | out | circle / ellipse / parabola / hyperbola / degenerate / empty | , |

**Applies when:** A, B, C not all 0; exact on the written coefficients; 'degenerate' = a point, a pair of lines or a degenerate equation (zero 3x3 determinant); 'empty' = an ellipse with no real points.

```
$ geometry solve conic_classify A=1 B=0 C=4 D=0 E=0 F=-4
type = ellipse
  by: Classify a conic section from its general equation  (discriminant B^2 - 4AC and the
3x3 determinant of the conic's matrix, exact in fractions)
  A = 1
  B = 0
  C = 4
  D = 0
  E = 0
  F = -4
  assumes: A, B, C not all 0; exact on the written coefficients; 'degenerate' = a point, a
pair of lines or a degenerate equation (zero 3x3 determinant); 'empty' = an ellipse with no
real points
```

**What the worked example means:** This calculation determines the geometric nature of the conic section defined by the general quadratic equation. It evaluates the discriminant using the coefficients of the squared terms, specifically 1, 0, and 4, along with the determinant of the associated matrix formed by all six coefficients including the linear terms 0 and 0 and the constant -4. The resulting classification is ellipse, which indicates whether the curve is a standard ellipse, parabola, hyperbola, or a degenerate case such as a point or pair of lines.

**Perimeter of an ellipse (exact, by the complete elliptic integral),** `ellipse_perimeter`

The exact perimeter (circumference) and area of an ellipse with semi-axes a and b.

$$P = 4a\,E(e), \quad E(m) = \frac{\pi}{2\,\mathrm{AGM}(1, \sqrt{1-m})}\left(1 - \sum_{n \geq 0} 2^{n-1} c_n^2\right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | one semi-axis | , |
| `b` | in | the other semi-axis | , |
| `perimeter` | out | 4 a E(e^2), a the larger semi-axis | , |
| `area` | out | pi a b | , |

**Applies when:** a, b > 0; axis ratio at least 0.001.

```
$ geometry solve ellipse_perimeter a=5 b=3
perimeter = 25.52699886
area = 47.1238898
  by: Perimeter of an ellipse (exact, by the complete elliptic integral)  (complete
elliptic integral of the second kind by the arithmetic-geometric mean (Gauss),
quadratically convergent)
  a = 5
  b = 3
  assumes: a, b > 0; axis ratio at least 0.001
```

**What the worked example means:** This calculation determines the precise circumference and enclosed region of an ellipse defined by semi-axes 5 and 3. The perimeter is derived using the complete elliptic integral of the second kind, ensuring accuracy beyond simple approximations, while the area follows directly from the product of the axes. The resulting perimeter is 25.527 and the area is 47.1239.

## Coordinate geometry (11)

### Angle between two lines, `angle_between_lines`

The acute (or right) angle between two lines given by equations.

$$\tan\theta = \left|\frac{a_1 b_2 - a_2 b_1}{a_1 a_2 + b_1 b_2}\right|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a1` | in | coefficient of x in line 1 (a1 x + b1 y + c1 = 0) | , |
| `b1` | in | coefficient of y in line 1 | , |
| `c1` | in | constant of line 1 | , |
| `a2` | in | coefficient of x in line 2 (a2 x + b2 y + c2 = 0) | , |
| `b2` | in | coefficient of y in line 2 | , |
| `c2` | in | constant of line 2 | , |
| `angle` | out | angle between the lines, degrees, 0 to 90 | , |

**Applies when:** each line has a or b nonzero; the acute angle is reported (parallel lines: 0).

```
$ geometry solve angle_between_lines a1=-1 b1=1 c1=-1 a2=1 b2=1 c2=-3
angle = 90
  by: Angle between two lines  (directions (b, -a); angle = atan2(|cross|, |dot|))
  a1 = -1
  b1 = 1
  c1 = -1
  a2 = 1
  b2 = 1
  c2 = -3
  assumes: each line has a or b nonzero; the acute angle is reported (parallel lines: 0)
```

**What the worked example means:** This calculation determines the acute angle formed by two straight lines defined by their general equations. It uses the coefficients of x and y from each line to find the angle, ignoring the constant terms which do not affect direction. The result 90 represents the smallest angle between the two lines, ranging from zero for parallel lines to ninety degrees for perpendicular ones.

**Barycentric coordinates of a point in a triangle, `barycentric_coordinates`**

The barycentric coordinates (l1, l2, l3) of a point with respect to triangle ABC: P = l1 A + l2 B + l3 C, l1 + l2 + l3 = 1.

$$\lambda_1 = \frac{[PBC]}{[ABC]},\ \lambda_2 = \frac{[APC]}{[ABC]},\ \lambda_3 = \frac{[ABP]}{[ABC]}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `points` | in | the triangle's vertices A, B, C | , |
| `l1` | out | weight of A | , |
| `l2` | out | weight of B | , |
| `l3` | out | weight of C | , |

**Applies when:** A, B, C not collinear; all weights positive exactly when P is inside.

```
$ geometry solve barycentric_coordinates px=1 py=1 "points=[0, 0, 4, 0, 0, 4]"
l1 = 0.5
l2 = 0.25
l3 = 0.25
  by: Barycentric coordinates of a point in a triangle  (ratios of signed areas: l1 =
[PBC]/[ABC], l2 = [APC]/[ABC], l3 = [ABP]/[ABC])
  px = 1
  py = 1
  assumes: A, B, C not collinear; all weights positive exactly when P is inside
```

**What the worked example means:** This calculation determines the barycentric coordinates of a specific point relative to a triangle defined by its three vertices. The result consists of three weights, labeled as the weight of vertex A, the weight of vertex B, and the weight of vertex C. These values, 0.5, 0.25, and 0.25, sum to one and indicate how much influence each vertex has on the position of the point. If all three weights are positive, the point lies strictly inside the triangle; otherwise, it is outside or on an edge.

### Circumcircle: the circle through three points, `triangle_circumcircle`

Circumcentre and circumradius of a triangle given by its vertices (the circle through three points).

$$|O - A| = |O - B| = |O - C| = R$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the three points: x1 y1, x2 y2, x3 y3 | , |
| `cx` | out | x of the circumcentre | , |
| `cy` | out | y of the circumcentre | , |
| `radius` | out | circumradius | , |

**Applies when:** exactly 3 points; not collinear.

```
$ geometry solve triangle_circumcircle "points=[0, 0, 4, 0, 0, 3]"
cx = 2
cy = 1.5
radius = 2.5
  by: Circumcircle: the circle through three points  (intersection of the perpendicular
bisectors, solved in coordinates relative to the first point)
  assumes: exactly 3 points; not collinear
```

**What the worked example means:** This calculation determines the center and size of the unique circle that passes through three given points. Solving for the intersection of perpendicular bisectors relative to the first point gives the circumcentre coordinates as 2 and 1.5. The distance from this centre to any vertex gives the circumradius of 2.5, defining the circle's extent.

### Incircle of a triangle: incentre and inradius, `triangle_incircle`

Incentre and inradius of the triangle with the given vertices.

$$I = \frac{aA + bB + cC}{a + b + c}, \quad r = \frac{2K}{a + b + c}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the vertices: xA yA, xB yB, xC yC | , |
| `ix` | out | x of the incentre | , |
| `iy` | out | y of the incentre | , |
| `radius` | out | inradius | , |

**Applies when:** exactly 3 points; not collinear.

```
$ geometry solve triangle_incircle "points=[0, 0, 4, 0, 0, 3]"
ix = 1
iy = 1
radius = 1
  by: Incircle of a triangle: incentre and inradius  (incentre = (a A + b B + c C)/(a + b +
c) with a, b, c the opposite side lengths; r = 2K/(a + b + c))
  assumes: exactly 3 points; not collinear
```

**What the worked example means:** This calculation determines the center and size of the circle inscribed within the triangle formed by the given vertices. The incentre, located at coordinates (1, 1), is equidistant from all three sides, while the inradius, measured as 1, represents that uniform distance. These values define the largest possible circle that fits entirely inside the triangle without crossing any boundaries.

### Intersection of a circle and a line, `circle_line_intersection`

The points where the line a x + b y + c = 0 meets the circle with centre (cx, cy) and radius r (0, 1 or 2 points).

$$d = \frac{|ac_x + bc_y + c|}{\sqrt{a^2 + b^2}}, \quad P = F \pm \sqrt{r^2 - d^2}\,\frac{(-b, a)}{\sqrt{a^2 + b^2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `cx` | in | x of the centre | , |
| `cy` | in | y of the centre | , |
| `r` | in | radius | , |
| `a` | in | coefficient of x of the line | , |
| `b` | in | coefficient of y of the line | , |
| `c` | in | constant of the line | , |
| `count` | out | number of intersection points (0, 1 or 2) | , |
| `x1` | out | x of the first point (ordered by x, then y) | , |
| `y1` | out | y of the first point | , |
| `x2` | out | x of the second point | , |
| `y2` | out | y of the second point | , |

**Applies when:** r > 0; a and b not both 0; tangency is decided exactly on the written values; points ordered by x, then y; absent points are empty.

```
$ geometry solve circle_line_intersection cx=0 cy=0 r=5 a=-1 b=1 c=-1
count = 2
x1 = -4
y1 = -3
x2 = 3
y2 = 4
  by: Intersection of a circle and a line  (foot of the perpendicular from the centre, then
the half-chord sqrt(r^2 - d^2) along the line)
  cx = 0
  cy = 0
  r = 5
  a = -1
  b = 1
  c = -1
  assumes: r > 0; a and b not both 0; tangency is decided exactly on the written values;
points ordered by x, then y; absent points are empty
```

In the worked example, cx = 0, cy = 0, r = 5, a = -1, b = 1, c = -1 go through the equation above and give count = 2, x1 = -4, y1 = -3, x2 = 3. It applies only when r > 0; a and b not both 0; tangency is decided exactly on the written values; points ordered by x, then y; absent points are empty: outside that it says what is wrong instead of returning a number.

**Intersection of two circles, `circle_circle_intersection`**

The points where two circles meet (0, 1 or 2 points).

$$a = \frac{d^2 + r_1^2 - r_2^2}{2d}, \quad h = \sqrt{r_1^2 - a^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `x1` | in | x of the first centre | , |
| `y1` | in | y of the first centre | , |
| `r1` | in | first radius | , |
| `x2` | in | x of the second centre | , |
| `y2` | in | y of the second centre | , |
| `r2` | in | second radius | , |
| `count` | out | number of intersection points (0, 1 or 2) | , |
| `x1` | out | x of the first point (ordered by x, then y) | , |
| `y1` | out | y of the first point | , |
| `x2` | out | x of the second point | , |
| `y2` | out | y of the second point | , |

**Applies when:** r1, r2 > 0; the circles are not identical; tangency decided exactly on the written values; points ordered by x, then y.

```
$ geometry solve circle_circle_intersection x1=0 y1=0 r1=5 x2=6 y2=0 r2=5
count = 2
x1 = 3
y1 = -4
x2 = 3
y2 = 4
  by: Intersection of two circles  (radical line: a = (d^2 + r1^2 - r2^2)/(2d) along the
centre line, h = sqrt(r1^2 - a^2) across it)
  x1 = 0
  y1 = 0
  r1 = 5
  x2 = 6
  y2 = 0
  r2 = 5
  assumes: r1, r2 > 0; the circles are not identical; tangency decided exactly on the
written values; points ordered by x, then y
```

**What the worked example means:** This calculation determines where two distinct circles meet by finding their radical line. Using the center distance and radii, it computes a projection length along the line connecting the centers and a perpendicular height. The result shows that there are 2 intersection points, with the first one found at position (3, -4). This method relies on the geometric property that all intersection points lie on a common chord perpendicular to the line joining the centers.

### Intersection point of two lines, `line_intersection`

The point where two lines a1 x + b1 y + c1 = 0 and a2 x + b2 y + c2 = 0 meet.

$$x = \frac{b_1 c_2 - b_2 c_1}{a_1 b_2 - a_2 b_1}, \quad y = \frac{c_1 a_2 - c_2 a_1}{a_1 b_2 - a_2 b_1}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a1` | in | coefficient of x in line 1 (a1 x + b1 y + c1 = 0) | , |
| `b1` | in | coefficient of y in line 1 | , |
| `c1` | in | constant of line 1 | , |
| `a2` | in | coefficient of x in line 2 (a2 x + b2 y + c2 = 0) | , |
| `b2` | in | coefficient of y in line 2 | , |
| `c2` | in | constant of line 2 | , |
| `x` | out | x of the intersection | , |
| `y` | out | y of the intersection | , |

**Applies when:** each line has a or b nonzero; the lines are not parallel (exact test on the written coefficients).

```
$ geometry solve line_intersection a1=2 b1=1 c1=-5 a2=1 b2=-1 c2=-1
x = 2
y = 1
  by: Intersection point of two lines  (Cramer's rule: x = (b1 c2 - b2 c1)/D, y = (c1 a2 -
c2 a1)/D, D = a1 b2 - a2 b1)
  a1 = 2
  b1 = 1
  c1 = -5
  a2 = 1
  b2 = -1
  c2 = -1
  assumes: each line has a or b nonzero; the lines are not parallel (exact test on the
written coefficients)
```

**What the worked example means:** This calculation determines the exact coordinates where two distinct lines meet. Using the coefficients for line one and line two, the method applies Cramer's rule to solve the system of linear equations. The resulting x coordinate is 2 and the y coordinate is 1, representing the single point common to both lines.

### Length of a polyline (path through points), `polyline_length`

The total length of the open path that visits the points in the order given.

$$L = \sum_{i=1}^{n-1} \|p_{i+1} - p_i\|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the points in order: x1 y1, x2 y2, ... | , |
| `length` | out | sum of the segment lengths | , |
| `segments` | out | number of segments | , |

**Applies when:** at least 2 points; the path is open (not closed back to the start).

```
$ geometry solve polyline_length "points=[0, 0, 3, 4, 3, 10]"
length = 11
segments = 2
  by: Length of a polyline (path through points)  (sum of the distances between consecutive
points)
  assumes: at least 2 points; the path is open (not closed back to the start)
```

**What the worked example means:** This calculation determines the cumulative distance along an open path defined by a sequence of points. It works by measuring the straight-line distance between each consecutive pair of vertices and adding these individual segment lengths together. The result, 11, represents the total travel distance required to visit all points in the specified order without returning to the start.

### Orthocentre of a triangle, `triangle_orthocenter`

The orthocentre (common point of the three altitudes) of the triangle with the given vertices.

$$(H - A) \cdot (C - B) = 0, \quad (H - B) \cdot (C - A) = 0$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the vertices: xA yA, xB yB, xC yC | , |
| `hx` | out | x of the orthocentre | , |
| `hy` | out | y of the orthocentre | , |

**Applies when:** exactly 3 points; not collinear.

```
$ geometry solve triangle_orthocenter "points=[0, 0, 4, 0, 1, 3]"
hx = 1
hy = 1
  by: Orthocentre of a triangle  (solve (H - A).(C - B) = 0 and (H - B).(C - A) = 0 (two
altitudes) by Cramer's rule)
  assumes: exactly 3 points; not collinear
```

**What the worked example means:** This calculation determines the orthocentre, which is the unique point where all three altitudes of a triangle intersect. The result is derived from the coordinates of the three non-collinear vertices provided as inputs. By solving the system of linear equations formed by two altitudes perpendicular to their respective opposite sides, the method yields the precise location of this geometric center. The computed horizontal coordinate is 1 and the vertical coordinate is 1, representing the position where the internal or external altitude lines meet.

**Tangents from a point to a circle, `tangent_from_point`**

The tangent length from an external point to a circle and the two points of tangency.

$$L = \sqrt{|P - C|^2 - r^2}, \quad T = C + \frac{r^2}{d^2}(P - C) \pm \frac{rL}{d^2}(P - C)^{\perp}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `cx` | in | x of the centre | , |
| `cy` | in | y of the centre | , |
| `r` | in | radius | , |
| `px` | in | x of the external point | , |
| `py` | in | y of the external point | , |
| `length` | out | tangent length sqrt(d^2 - r^2) | , |
| `x1` | out | x of the first tangency point (ordered by x, then y) | , |
| `y1` | out | y of the first tangency point | , |
| `x2` | out | x of the second tangency point | , |
| `y2` | out | y of the second tangency point | , |

**Applies when:** r > 0; the point is not inside the circle (on the circle: length 0, both points at the point).

```
$ geometry solve tangent_from_point cx=0 cy=0 r=3 px=5 py=0
length = 4
x1 = 1.8
y1 = -2.4
x2 = 1.8
y2 = 2.4
  by: Tangents from a point to a circle  (L = sqrt(d^2 - r^2); T = C + (r^2/d^2)(P - C) +-
(r L/d^2) perp(P - C))
  cx = 0
  cy = 0
  r = 3
  px = 5
  py = 0
  assumes: r > 0; the point is not inside the circle (on the circle: length 0, both points
at the point)
```

**What the worked example means:** This calculation determines the distance from an external point to a circle along its tangent lines, as well as the specific coordinates where these lines touch the circle. Given a centre at (0, 0) with radius 3 and an external point at (5, 0), the system computes the tangent length as 4. The resulting points of contact are located at (1.8, -2.4) for the first tangent and (1.8, 2.4) for the second, ordered by their x-coordinates.

**Triangle from vertex coordinates: area, perimeter, centroid, angles, `triangle_coordinates`**

Area, perimeter, centroid and the angles at A, B, C of the triangle with vertices A, B, C.

$$K = \tfrac{1}{2}\left|(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)\right|, \quad G = \tfrac{1}{3}(A + B + C)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the vertices A, B, C: xA yA, xB yB, xC yC | , |
| `area` | out | area (shoelace) | , |
| `perimeter` | out | perimeter | , |
| `gx` | out | x of the centroid | , |
| `gy` | out | y of the centroid | , |
| `angle_A` | out | angle at A, degrees | , |
| `angle_B` | out | angle at B, degrees | , |
| `angle_C` | out | angle at C, degrees | , |

**Applies when:** exactly 3 vertices; not collinear.

```
$ geometry solve triangle_coordinates "points=[0, 0, 4, 0, 0, 3]"
area = 6
perimeter = 12
gx = 1.333333333
gy = 1
angle_A = 90
angle_B = 36.86989765
angle_C = 53.13010235
  by: Triangle from vertex coordinates: area, perimeter, centroid, angles  (shoelace area;
centroid = mean of the vertices; angles by the law of cosines as atan2(4K, b^2 + c^2 -
a^2))
  assumes: exactly 3 vertices; not collinear
```

**What the worked example means:** This calculation determines the geometric properties of a triangle defined by three vertices. Using the shoelace formula, the enclosed area is found to be 6. The total distance around the shape, or perimeter, is calculated as 12. The centroid, representing the average position of the three vertices, is located at x-coordinate 1.33333 and y-coordinate 1. Additionally, the internal angles at each vertex are computed using vector dot products, resulting in angle A being 90, angle B being 36.8699, and angle C being 53.1301. These values provide a complete spatial description of the triangle's size, shape, and orientation.

## Curves (4)

### Arc length of a parabola, `parabola_arc_length`

The length of the curve y = a x^2 + b x + c between x = x0 and x = x1.

$$L = \frac{1}{2a}\left[\tfrac{1}{2}\left(u\sqrt{1+u^2} + \operatorname{arsinh} u\right)\right]_{u_0}^{u_1}, \quad u = 2ax + b$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | coefficient of x^2 | , |
| `b` | in | coefficient of x | , |
| `x0` | in | start x | , |
| `x1` | in | end x | , |
| `c` | in | constant (does not affect the length) (optional) | , |
| `length` | out | arc length | , |

**Applies when:** x0 != x1 (either order).

```
$ geometry solve parabola_arc_length a=1 b=0 c=0 x0=0 x1=1
length = 1.478942858
  by: Arc length of a parabola  (closed form of the integral of sqrt(1 + u^2), u = 2ax + b:
(u sqrt(1+u^2) + asinh u)/2, divided by 2a)
  a = 1
  b = 0
  c = 0
  x0 = 0
  x1 = 1
  assumes: x0 != x1 (either order)
```

**What the worked example means:** This calculation determines the precise distance along the curved path of a quadratic function defined by coefficients 1, 0, and 0. The result, 1.47894, represents the total length of the curve segment starting at horizontal position 0 and ending at horizontal position 1. Unlike straight-line distance, this measure accounts for the curvature introduced by the squared term, which is essential for applications requiring exact path lengths such as cable routing or optical design.

**Curvature of a polynomial graph at a point, `curvature_of_graph`**

The curvature and radius of curvature of y = p(x) at x = x0 for a polynomial p.

$$\kappa = \frac{|y''|}{(1 + y'^2)^{3/2}}, \quad R = \frac{1}{\kappa}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `coefficients` | in | coefficients from the highest power down: a_n, ..., a_1, a_0 | , |
| `x0` | in | the x-coordinate of the point | , |
| `curvature` | out | \|y''\| / (1 + y'^2)^(3/2) | , |
| `radius_of_curvature` | out | 1 / curvature (none where the curvature is 0) | , |
| `slope` | out | y'(x0) | , |

**Applies when:** polynomial of degree at most 10.

```
$ geometry solve curvature_of_graph "coefficients=[1, 0, 0]" x0=0
curvature = 2
radius_of_curvature = 0.5
slope = 0
  by: Curvature of a polynomial graph at a point  (exact derivatives of the polynomial
(Horner), then k = |y''|/(1 + y'^2)^(3/2))
  x0 = 0
  assumes: polynomial of degree at most 10
```

**What the worked example means:** This calculation determines the curvature and radius of curvature for the polynomial graph defined by [1, 0, 0] at the specific x-coordinate 0. The result, 2, quantifies how sharply the curve bends at that point, while 0 indicates the steepness of the tangent line. These values help engineers and designers understand the geometric behavior of the curve in local regions.

**Curvature of an ellipse at a point, `ellipse_curvature`**

The curvature and radius of curvature of the ellipse (a cos t, b sin t) at parameter t (eccentric anomaly).

$$\kappa(t) = \frac{ab}{(a^2 \sin^2 t + b^2 \cos^2 t)^{3/2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | semi-axis along x | , |
| `b` | in | semi-axis along y | , |
| `t` | in | parameter t (eccentric anomaly), degrees | , |
| `curvature` | out | ab / (a^2 sin^2 t + b^2 cos^2 t)^(3/2) | , |
| `radius_of_curvature` | out | 1 / curvature | , |
| `x` | out | x of the point | , |
| `y` | out | y of the point | , |

**Applies when:** a, b > 0; the point is (a cos t, b sin t); 'end of the major/minor axis' means t = 0 / 90 degrees.

```
$ geometry solve ellipse_curvature a=5 b=3 t=0
curvature = 0.5555555556
radius_of_curvature = 1.8
x = 5
y = 0
  by: Curvature of an ellipse at a point  (parametric curvature |x'y'' - y'x''| / (x'^2 +
y'^2)^(3/2) in closed form)
  a = 5
  b = 3
  t = 0
  assumes: a, b > 0; the point is (a cos t, b sin t); 'end of the major/minor axis' means t
= 0 / 90 degrees
```

**What the worked example means:** This calculation determines how sharply the ellipse bends at a specific location defined by the eccentric anomaly. Using the semi-axis along x, 5, and the semi-axis along y, 3, evaluated at parameter t, 0, the model computes the curvature as 0.555556. The corresponding radius of curvature, which represents the size of the circle that best approximates the curve at that point, is found to be 1.8. The coordinates of this point on the ellipse are given as x equal to 5 and y equal to 0. These values describe the local geometric behavior of the elliptical path.

**Helix: curvature, torsion and length per turn, `helix_measures`**

Curvature, torsion and arc length per turn of the circular helix of radius r and pitch p (rise per turn).

$$\kappa = \frac{r}{r^2 + c^2}, \quad \tau = \frac{c}{r^2 + c^2}, \quad c = \frac{p}{2\pi}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius r of the helix | , |
| `pitch` | in | pitch p: rise per full turn (negative: left-handed) | , |
| `curvature` | out | r / (r^2 + c^2), c = p / (2 pi) | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `torsion` | out | c / (r^2 + c^2) | , |
| `length_per_turn` | out | sqrt((2 pi r)^2 + p^2) | , |

**Applies when:** radius > 0; parametrised (r cos t, r sin t, c t).

```
$ geometry solve helix_measures radius=1 pitch=6.283185307
curvature = 0.5
torsion = 0.5
length_per_turn = 8.885765876
  by: Helix: curvature, torsion and length per turn  (Frenet-Serret closed forms for the
circular helix)
  radius = 1
  pitch = 6.283185307
  assumes: radius > 0; parametrised (r cos t, r sin t, c t)
```

**What the worked example means:** This calculation determines the geometric properties of a circular helix defined by radius 1 and pitch 6.28319. Using the Frenet-Serret formulas, it computes the curvature as 0.5, which measures how sharply the curve bends. The torsion is calculated as 0.5, indicating the rate at which the curve twists out of its osculating plane. Finally, the arc length for one complete revolution is found to be 8.88577. These values are essential for understanding the physical shape and mechanical behavior of spiral structures.

## Plane figures (10)

### Area of a triangle from base and height, `triangle_area_base_height`

The area of a triangle from a base and the height (altitude) to that base.

$$A = \tfrac{1}{2} b h$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `base` | in | length of the base | , |
| `height` | in | height perpendicular to the base | , |
| `area` | out | area b*h/2 | , |

**Applies when:** base > 0; height > 0.

```
$ geometry solve triangle_area_base_height base=10 height=6
area = 30
  by: Area of a triangle from base and height  (A = b h / 2)
  base = 10
  height = 6
  assumes: base > 0; height > 0
```

**What the worked example means:** This calculation finds the area of a triangle using the length of its base and the perpendicular height. Taking half the product of the base and the height shows that the enclosed space measures 30. This result represents the total two-dimensional surface covered by the triangular shape.

### Area of an annulus (ring), `annulus_area`

The area between two concentric circles.

$$A = \pi(R^2 - r^2)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `outer` | in | outer radius | , |
| `inner` | in | inner radius | , |
| `area` | out | pi (R^2 - r^2) | , |

**Applies when:** 0 < inner < outer.

```
$ geometry solve annulus_area outer=5 inner=3
area = 50.26548246
  by: Area of an annulus (ring)  (A = pi (R - r)(R + r))
  outer = 5
  inner = 3
  assumes: 0 < inner < outer
```

**What the worked example means:** This calculation finds the area of the annulus, which is the region between two concentric circles. By using the outer radius of 5 and the inner radius of 3, the model determines that the total surface area of this ring-shaped shape is 50.2655. This result represents the amount of space enclosed by the larger circle minus the space occupied by the smaller central circle.

### Circle: area, circumference and diameter, `circle_measures`

Area, circumference and diameter of a circle from its radius (or diameter).

$$A = \pi r^2, \quad C = 2\pi r$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius of the circle | , |
| `area` | out | area pi r^2 | , |
| `circumference` | out | circumference 2 pi r | , |
| `diameter` | out | diameter 2r | , |

**Applies when:** radius > 0.

```
$ geometry solve circle_measures radius=3
area = 28.27433388
circumference = 18.84955592
diameter = 6
  by: Circle: area, circumference and diameter  (A = pi r^2, C = 2 pi r)
  radius = 3
  assumes: radius > 0
```

**What the worked example means:** This calculation determines the fundamental geometric properties of a circle starting from its radius. Using the input value 3, it computes the total surface enclosed by the boundary, resulting in an area of 28.2743. It also calculates the length of the perimeter, yielding a circumference of 18.8496, and doubles the radius to find the diameter, which is 6. These metrics are essential for understanding the size and extent of circular shapes.

**Circular sector: arc length, area and chord, `sector_measures`**

Arc length, sector area and chord of a circular sector from the radius and central angle.

$$s = r\theta, \quad A = \tfrac{1}{2} r^2 \theta, \quad c = 2r \sin \tfrac{\theta}{2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius | , |
| `angle` | in | central angle in degrees | , |
| `arc_length` | out | r theta (theta in radians) | , |
| `sector_area` | out | r^2 theta / 2 | , |
| `chord` | out | 2 r sin(theta/2) | , |

**Applies when:** radius > 0; 0 < angle <= 360 degrees.

```
$ geometry solve sector_measures radius=6 angle=60
arc_length = 6.283185307
sector_area = 18.84955592
chord = 6
  by: Circular sector: arc length, area and chord  (theta = angle * pi/180; s = r theta; A
= r^2 theta / 2; chord = 2 r sin(theta/2))
  radius = 6
  angle = 60
  assumes: radius > 0; 0 < angle <= 360 degrees
```

**What the worked example means:** This calculation determines three key properties of a circular sector using the provided radius and central angle. First, the angle is converted from degrees to radians to apply standard geometric formulas. The resulting arc length, representing the curved edge distance, is 6.28319. The sector area, which measures the surface enclosed by the two radii and the arc, is calculated as 18.8496. Finally, the chord length, defined as the straight-line distance connecting the endpoints of the arc, is found to be 6. These values collectively describe the geometry of the slice formed by the given parameters.

**Circular segment: area, chord, height, arc, `circular_segment`**

Area, chord, height (sagitta) and arc length of the circular segment cut off by a chord with a given central angle.

$$A = \tfrac{1}{2} r^2 (\theta - \sin\theta), \quad c = 2r \sin \tfrac{\theta}{2}, \quad h = r\left(1 - \cos \tfrac{\theta}{2}\right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `angle` | in | central angle in degrees | , |
| `area` | out | r^2 (theta - sin theta)/2 | , |
| `chord` | out | 2 r sin(theta/2) | , |
| `height` | out | r (1 - cos(theta/2)) | , |
| `arc_length` | out | r theta | , |

**Applies when:** radius > 0; 0 < angle < 360 degrees.

```
$ geometry solve circular_segment radius=10 angle=60
area = 9.058607371
chord = 10
height = 1.339745962
arc_length = 10.47197551
  by: Circular segment: area, chord, height, arc  (A = r^2 (theta - sin theta)/2 (series
for small theta to avoid cancellation); h = 2 r sin^2(theta/4))
  radius = 10
  angle = 60
  assumes: radius > 0; 0 < angle < 360 degrees
```

**What the worked example means:** This calculation determines the geometric properties of a circular segment defined by a radius of 10 and a central angle of 60. The resulting area is 9.05861, representing the region enclosed between the chord and the arc. The straight-line distance across the segment, known as the chord, measures 10. The maximum depth of the segment from the chord to the arc, or sagitta, is 1.33975. Finally, the curved boundary along the circle's circumference, called the arc length, spans 10.472. These values provide a complete description of the segment's size and shape based on the given inputs.

### Classify a triangle by its sides and angles, `triangle_classify`

Whether a triangle with sides a, b, c is equilateral, isosceles or scalene, and acute, right or obtuse.

$$x^2 + y^2 \gtreqless z^2 \;\Leftrightarrow\; \text{acute} \,/\, \text{right} \,/\, \text{obtuse}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | side a | , |
| `b` | in | side b | , |
| `c` | in | side c | , |
| `by_sides` | out | equilateral / isosceles / scalene | , |
| `by_angles` | out | acute / right / obtuse | , |

**Applies when:** a, b, c > 0; triangle inequality; the sides are compared exactly as written.

```
$ geometry solve triangle_classify a=3 b=4 c=5
by_sides = scalene
by_angles = right
  by: Classify a triangle by its sides and angles  (exact comparison: with z the longest
side, x^2 + y^2 - z^2 > 0 acute, = 0 right, < 0 obtuse (converse of Pythagoras))
  a = 3
  b = 4
  c = 5
  assumes: a, b, c > 0; triangle inequality; the sides are compared exactly as written
```

**What the worked example means:** This calculation determines the geometric classification of a triangle based on its three side lengths. First, it compares the sides to see if they are all equal, two are equal, or none are equal, resulting in the classification scalene. Next, it identifies the longest side and applies the converse of the Pythagorean theorem by comparing the sum of the squares of the two shorter sides against the square of the longest side. This comparison yields the angular classification right, indicating whether all angles are less than ninety degrees, one angle is exactly ninety degrees, or one angle exceeds ninety degrees.

**Medians, altitudes and angle bisectors of a triangle, `triangle_cevians`**

The lengths of the three medians, three altitudes and three internal angle bisectors of a triangle from its sides.

$$m_a = \tfrac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}, \quad h_a = \frac{2K}{a}, \quad t_a = \frac{\sqrt{bc\,[(b+c)^2 - a^2]}}{b+c}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | side a (opposite A) | , |
| `b` | in | side b (opposite B) | , |
| `c` | in | side c (opposite C) | , |
| `median_a` | out | median from vertex A (to side a) | , |
| `median_b` | out | median from vertex B (to side b) | , |
| `median_c` | out | median from vertex C (to side c) | , |
| `altitude_a` | out | altitude from vertex A (to side a) | , |
| `altitude_b` | out | altitude from vertex B (to side b) | , |
| `altitude_c` | out | altitude from vertex C (to side c) | , |
| `bisector_a` | out | bisector from vertex A (to side a) | , |
| `bisector_b` | out | bisector from vertex B (to side b) | , |
| `bisector_c` | out | bisector from vertex C (to side c) | , |

**Applies when:** a, b, c > 0; triangle inequality.

```
$ geometry solve triangle_cevians a=3 b=4 c=5
median_a = 4.272001873
median_b = 3.605551275
median_c = 2.5
altitude_a = 4
altitude_b = 3
altitude_c = 2.4
bisector_a = 4.216370214
bisector_b = 3.354101966
bisector_c = 2.424366107
  by: Medians, altitudes and angle bisectors of a triangle  (m_a = sqrt(2b^2 + 2c^2 -
a^2)/2; h_a = 2K/a (Heron); t_a = sqrt(bc((b + c)^2 - a^2))/(b + c))
  a = 3
  b = 4
  c = 5
  assumes: a, b, c > 0; triangle inequality
```

**What the worked example means:** This calculation determines the lengths of key internal lines for a triangle defined by side lengths 3, 4, and 5. The median from vertex A, which connects to the midpoint of side a, measures 4.272. The altitude from vertex A, representing the perpendicular distance to side a, is 4. The internal angle bisector from vertex A, dividing the angle at that vertex into two equal parts, has a length of 4.21637. These values help characterize the geometric structure and proportions of the triangle.

**Rectangle: area, perimeter and diagonal, `rectangle_measures`**

Area, perimeter and diagonal of a rectangle (a square when length = width).

$$A = lw, \quad P = 2(l + w), \quad d = \sqrt{l^2 + w^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `length` | in | length of the rectangle | , |
| `width` | in | width of the rectangle | , |
| `area` | out | area l*w | , |
| `perimeter` | out | perimeter 2(l + w) | , |
| `diagonal` | out | diagonal sqrt(l^2 + w^2) | , |

**Applies when:** length > 0; width > 0.

```
$ geometry solve rectangle_measures length=8 width=5
area = 40
perimeter = 26
diagonal = 9.433981132
  by: Rectangle: area, perimeter and diagonal  (A = l w, P = 2(l + w), d = sqrt(l^2 + w^2)
(Pythagoras))
  length = 8
  width = 5
  assumes: length > 0; width > 0
```

**What the worked example means:** This calculation determines three key measurements for a rectangle defined by its length and width. The area, representing the surface enclosed within the shape, is found to be

40. The perimeter, which is the total distance around the boundary of the rectangle, equals 26. Additionally, the diagonal, measuring the straight-line distance between opposite corners, is calculated as 9.43398. These values provide a complete geometric summary of the rectangular space based on the given dimensions.

**Rhombus from its diagonals: area, side, perimeter, `rhombus_measures`**

Area, side and perimeter of a rhombus from its two diagonals.

$$A = \tfrac{1}{2} d_1 d_2, \quad s = \tfrac{1}{2}\sqrt{d_1^2 + d_2^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `d1` | in | first diagonal | , |
| `d2` | in | second diagonal | , |
| `area` | out | d1 d2 / 2 | , |
| `side` | out | sqrt(d1^2 + d2^2)/2 | , |
| `perimeter` | out | 4 * side | , |

**Applies when:** d1, d2 > 0 (the diagonals of a rhombus bisect each other at right angles).

```
$ geometry solve rhombus_measures d1=6 d2=8
area = 24
side = 5
perimeter = 20
  by: Rhombus from its diagonals: area, side, perimeter  (A = d1 d2 / 2; the half-diagonals
are the legs of a right triangle whose hypotenuse is a side)
  d1 = 6
  d2 = 8
  assumes: d1, d2 > 0 (the diagonals of a rhombus bisect each other at right angles)
```

**What the worked example means:** This calculation determines the geometric properties of a rhombus using its two diagonal lengths, 6 and 8. The area is found by taking half the product of the diagonals, resulting in 24. Since the diagonals bisect each other at right angles, they form four congruent right triangles where the legs are half the diagonal lengths. Using the Pythagorean theorem on these legs yields the side length 5. Finally, because all sides of a rhombus are equal, multiplying the side length by four gives the total perimeter 20.

**Trapezoid: area and midsegment, `trapezoid_measures`**

Area and midsegment (median) of a trapezoid from its parallel sides and height.

$$A = \frac{(a+b)\,h}{2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | first parallel side | , |
| `b` | in | second parallel side | , |
| `height` | in | distance between the parallel sides | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `area` | out | (a + b) h / 2 | , |
| `midsegment` | out | (a + b) / 2 | , |

**Applies when:** a, b, height > 0.

```
$ geometry solve trapezoid_measures height=4 a=6 b=10
area = 32
midsegment = 8
  by: Trapezoid: area and midsegment  (A = (a + b) h / 2)
  height = 4
  a = 6
  b = 10
  assumes: a, b, height > 0
```

**What the worked example means:** This calculation determines the area and midsegment of a trapezoid using its two parallel sides and height. The area is found by averaging the lengths of the parallel sides and multiplying by the perpendicular distance between them, resulting in 32. The midsegment, which connects the midpoints of the non-parallel sides, has a length equal to the average of the parallel sides, calculated here as 8.

## Polygons (7)

### Centroid of a polygon, `polygon_centroid`

The centroid (centre of mass of the uniform region) of a simple polygon given its vertices in order.

$$C_x = \frac{1}{6A}\sum_i (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | vertices in order: x1 y1, x2 y2, ... | , |
| `cx` | out | x of the centroid | , |
| `cy` | out | y of the centroid | , |

**Applies when:** at least 3 vertices, in order; simple polygon.

```
$ geometry solve polygon_centroid "points=[0, 0, 4, 0, 4, 2, 0, 2]"
cx = 2
cy = 1
  by: Centroid of a polygon  (area-weighted shoelace sums: Cx = sum((x_i + x_{i+1}) w_i) /
(3 * sum w_i), w_i = x_i y_{i+1} - x_{i+1} y_i)
  assumes: at least 3 vertices, in order; simple polygon
```

**What the worked example means:** This calculation determines the geometric center of a simple polygon using its ordered vertices. By applying area weighted sums, the method finds the average position of all points within the region, which acts as the center of mass for a uniform shape. The resulting coordinates are x equals 2 and y equals 1, representing the balance point derived from the input set [0, 0, 4, 0, 4, 2, 0, 2].

### Interior angles of a polygon from its vertices, `polygon_interior_angles`

The interior angle at every vertex of a simple polygon (reflex angles included), their sum and the number of reflex vertices.

$$\theta_i = 180^\circ - \operatorname{atan2}(\mathbf{u} \times \mathbf{v}, \mathbf{u} \cdot \mathbf{v}), \quad \sum \theta_i = (n-2)180^\circ$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | vertices in order: x1 y1, x2 y2, ... | , |
| `angles` | out | interior angle at each vertex, degrees, in the order given | , |
| `angle_sum` | out | (n - 2) 180 | , |
| `reflex_vertices` | out | number of angles above 180 degrees | , |

**Applies when:** simple polygon; either orientation.

```
$ geometry solve polygon_interior_angles "points=[0, 0, 2, 0, 2, 2, 1, 1, 0, 2]"
angles = [90.0,90.0,45.0,270.0,45.0]
angle_sum = 540
reflex_vertices = 1
  by: Interior angles of a polygon from its vertices  (turning angle at each vertex by
atan2 of the cross and dot products of consecutive edges, interior = 180 - turn
(orientation-corrected))
  assumes: simple polygon; either orientation
```

**What the worked example means:** This calculation determines the interior angle at each vertex of a simple polygon defined by its ordered points. By analyzing the turning direction between consecutive edges, it computes specific angle measures in degrees for every corner, listed as [90, 90, 45, 270, 45]. The total of these individual angles is reported as 540, which theoretically equals the sum of interior angles for an n-sided polygon. Additionally, the count of vertices where the internal angle exceeds one hundred eighty degrees, known as reflex vertices, is given by 1. This provides a complete geometric description of the shape's local curvature and global consistency.

### Is a polygon convex?, `polygon_is_convex`

Whether a simple polygon given by its vertices is convex.

$$\operatorname{sign}\left((p_{i+1} - p_i) \times (p_{i+2} - p_{i+1})\right) \text{ constant}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | vertices in order: x1 y1, x2 y2, ... | , |
| `convex` | out | yes / no | , |

**Applies when:** simple polygon; a vertex on a straight edge (180 degrees) does not make it non-convex; exact on the written coordinates.

```
$ geometry solve polygon_is_convex "points=[0, 0, 4, 0, 4, 4, 2, 1, 0, 4]"
convex = no
  by: Is a polygon convex?  (every turn (cross product of consecutive edges) has the same
sign, computed exactly)
  assumes: simple polygon; a vertex on a straight edge (180 degrees) does not make it non-
convex; exact on the written coordinates
```

**What the worked example means:** This calculation checks the geometric property of convexity for a simple polygon. It examines every consecutive set of three vertices from [0, 0, 4, 0, 4, 4, 2, 1, 0, 4] and computes the cross product of the two adjacent edge vectors. If the sign of this cross product remains constant across all turns, indicating that the polygon consistently bends in the same direction without any reflex angles greater than one hundred eighty degrees, the shape is classified as convex. The final determination is reported as no.

**Pick's theorem: lattice points of a polygon, `pick_lattice_points`**

Area, boundary lattice points and interior lattice points of a simple polygon with integer vertices.

$$A = I + \frac{B}{2} - 1$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | integer vertices in order: x1 y1, x2 y2, ... | , |
| `area` | out | area | , |
| `boundary_points` | out | lattice points on the boundary | , |
| `interior_points` | out | lattice points strictly inside | , |

**Applies when:** integer coordinates; simple polygon.

```
$ geometry solve pick_lattice_points "points=[0, 0, 4, 0, 4, 3, 0, 3]"
area = 12
boundary_points = 14
interior_points = 6
  by: Pick's theorem: lattice points of a polygon  (B = sum of gcd(|dx|, |dy|) over the
edges; shoelace area A; Pick: I = A - B/2 + 1)
  assumes: integer coordinates; simple polygon
```

**What the worked example means:** This calculation determines the geometric properties of a simple polygon defined by integer vertices. Using the shoelace formula, it computes the total area as 12. It also counts the lattice points lying exactly on the edges, resulting in 14 boundary points. Finally, applying Pick's theorem, which relates area to interior and boundary points, it derives that there are 6 strictly interior lattice points.

**Polygon angles and diagonals, `regular_polygon_angles`**

Each interior and exterior angle of a regular n-gon, the interior angle sum of any n-gon and its number of diagonals.

$$\theta_{int} = \frac{(n-2)\,180^\circ}{n}, \quad \theta_{ext} = \frac{360^\circ}{n}, \quad D = \frac{n(n-3)}{2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `n` | in | number of sides | , |
| `interior_angle` | out | each interior angle of the regular polygon, degrees | , |
| `exterior_angle` | out | each exterior angle, degrees | , |
| `interior_sum` | out | sum of the interior angles, degrees | , |
| `diagonals` | out | number of diagonals n(n-3)/2 | , |

**Applies when:** 3 <= n <= 1000 (whole number); each single angle assumes a regular polygon.

```
$ geometry solve regular_polygon_angles n=6
interior_angle = 120
exterior_angle = 60
interior_sum = 720
diagonals = 9
  by: Polygon angles and diagonals  (sum = (n - 2) 180; interior = sum / n; exterior = 360
/ n; diagonals = n(n - 3)/2)
  assumes: 3 <= n <= 1000 (whole number); each single angle assumes a regular polygon
```

**What the worked example means:** This calculation determines key geometric properties based on the number of sides, 6. It computes the total sum of interior angles as 720, which applies to any simple polygon. Assuming the shape is regular, it derives the measure of each individual interior angle as 120 and each exterior angle as 60. Additionally, it calculates the total number of diagonals connecting non-adjacent vertices, resulting in 9. These values provide a complete angular and structural summary for the specified n-gon.

**Polygon from vertices: area, perimeter, orientation, `polygon_area`**

Area (shoelace formula), perimeter and orientation of a simple polygon given its vertices in order.

$$A = \tfrac{1}{2}\left|\sum_{i}\left(x_i y_{i+1} - x_{i+1} y_i\right)\right|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | vertices in order: x1 y1, x2 y2, ... | , |
| `area` | out | enclosed area | , |
| `perimeter` | out | sum of the edge lengths | , |
| `orientation` | out | counterclockwise or clockwise | , |

**Applies when:** at least 3 vertices, in order; the polygon is simple (its edges do not cross).

```
$ geometry solve polygon_area "points=[0, 0, 4, 0, 4, 3, 0, 3]"
area = 12
perimeter = 14
orientation = counterclockwise
  by: Polygon from vertices: area, perimeter, orientation  (shoelace formula A = |sum(x_i
y_{i+1} - x_{i+1} y_i)| / 2; the sign gives the orientation; simplicity checked exactly)
  assumes: at least 3 vertices, in order; the polygon is simple (its edges do not cross)
```

**What the worked example means:** This calculation determines the geometric properties of a simple polygon using its ordered vertices. The enclosed area is found using the shoelace formula, which sums cross products of adjacent vertex coordinates to yield 12. The total boundary length, or perimeter, is calculated as the sum of distances between consecutive vertices, resulting in 14. Additionally, the sign of the coordinate sum indicates whether the vertices are listed in a counterclockwise or clockwise direction, identified here as counterclockwise. These values describe the size and shape configuration of the region bounded by the input points.

**Regular polygon: area, perimeter, apothem, circumradius,** `regular_polygon_measures`

Area, perimeter, apothem (inradius) and circumradius of a regular n-gon from its side length.

$$a = \frac{s}{2\tan(\pi/n)}, \quad R = \frac{s}{2\sin(\pi/n)}, \quad A = \tfrac{1}{2} n s a$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `n` | in | number of sides | , |
| `side` | in | side length | , |
| `area` | out | n s a / 2 | , |
| `perimeter` | out | n s | , |
| `apothem` | out | s / (2 tan(pi/n)) | , |
| `circumradius` | out | s / (2 sin(pi/n)) | , |

**Applies when:** 3 <= n <= 200; side > 0.

```
$ geometry solve regular_polygon_measures n=6 side=4
area = 41.56921938
perimeter = 24
apothem = 3.464101615
circumradius = 4
  by: Regular polygon: area, perimeter, apothem, circumradius  (apothem a = s/(2
tan(pi/n)), R = s/(2 sin(pi/n)), A = (perimeter * apothem)/2)
  side = 4
  assumes: 3 <= n <= 200; side > 0
```

**What the worked example means:** This calculation determines the geometric properties of a regular polygon based on its number of sides and side length. Given 6 sides and a side length of 4, the total boundary length, or perimeter, is computed as 24. The distance from the center to the midpoint of any side, known as the apothem, is found to be 3.4641, while the radius extending from the center to any vertex, called the circumradius, is 4. Using these internal measurements, the total surface area enclosed by the shape is calculated as 41.5692, representing the sum of all triangular sections formed by connecting the center to each vertex.

## Polyhedra (1)

**Platonic solids: counts, Euler characteristic, volume, surface, radii,** `platonic_solid`

Vertices, edges, faces, Euler characteristic, volume, surface area, circumradius and inradius of a regular polyhedron of edge a.

$$V - E + F = 2, \quad V_{\text{ico}} = \tfrac{5}{12}(3+\sqrt{5})\,a^3$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `solid` | in | which Platonic solid | , |
| `edge` | in | edge length a | , |
| `vertices` | out | V | , |
| `edges` | out | E | , |
| `faces` | out | F | , |
| `euler_characteristic` | out | V - E + F | , |
| `volume` | out | volume | , |
| `surface_area` | out | surface area | , |
| `circumradius` | out | radius of the circumscribed sphere | , |
| `inradius` | out | radius of the inscribed sphere | , |

**Applies when:** edge > 0.

```
$ geometry solve platonic_solid solid=icosahedron edge=2
vertices = 12
edges = 30
faces = 20
euler_characteristic = 2
volume = 17.45355993
surface_area = 34.64101615
circumradius = 1.902113033
inradius = 1.511522628
  by: Platonic solids: counts, Euler characteristic, volume, surface, radii  (closed forms
in the edge length (golden ratio for the dodecahedron and icosahedron); Euler: V - E + F =
2)
  edge = 2
  solid = icosahedron
  assumes: edge > 0
```

**What the worked example means:** This calculation determines the fundamental geometric properties of a specified Platonic solid based on its edge length. Using the provided edge size 2, it computes the exact counts for vertices 12, edges 30, and faces 20. These integer counts satisfy the topological constraint where the Euler characteristic is always 2. Additionally, the method derives continuous metrics including the volume 17.4536, surface area 34.641, circumradius 1.90211, and inradius 1.51152, providing a complete description of the polyhedron's spatial extent.

## Similarity (1)

### Similar figures: scale factor, missing lengths, area and volume ratios, `similar_figures`

For two similar figures, the scale factor from corresponding lengths (or areas, or volumes), the ratios of their areas and volumes, and the measurements of the second figure that correspond to a length, area or volume of the first.

$$k = \frac{l_2}{l_1}, \qquad \frac{A_2}{A_1} = k^2, \qquad \frac{V_2}{V_1} = k^3$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `measure` | in | what the two corresponding values measure: lengths, areas or volumes | , |
| `m1` | in | the value in the first figure | , |
| `m2` | in | the corresponding value in the second figure | , |
| `other_length` | in | another length of the first figure (optional) | , |
| `other_area` | in | an area of the first figure (optional) | , |
| `other_volume` | in | a volume of the first figure (optional) | , |
| `scale_factor` | out | the scale factor k (second : first) | , |
| `area_ratio` | out | ratio of areas k^2 | , |
| `volume_ratio` | out | ratio of volumes k^3 | , |
| `length2` | out | the corresponding length of the second figure | , |
| `area2` | out | the corresponding area of the second figure | , |
| `volume2` | out | the corresponding volume of the second figure | , |

**Applies when:** the values are positive; the figures are similar (one is a scaled copy of the other).

```
$ geometry solve similar_figures measure=lengths m1=4 m2=6
scale_factor = 1.5
area_ratio = 2.25
volume_ratio = 3.375
length2 = null
area2 = null
volume2 = null
  by: Similar figures: scale factor, missing lengths, area and volume ratios  (k = l2/l1
(or sqrt of the area ratio, or cube root of the volume ratio); lengths scale by k, areas by
k^2, volumes by k^3)
  m1 = 4
  m2 = 6
  measure = lengths
  assumes: the values are positive; the figures are similar (one is a scaled copy of the
other)
```

In the worked example, measure = lengths, m1 = 4, m2 = 6 go through the equation above and give scale_factor = 1.5, area_ratio = 2.25, volume_ratio = 3.375, length2 = None. It applies only when the values are positive; the figures are similar (one is a scaled copy of the other): outside that it says what is wrong instead of returning a number.

## Solid coordinate geometry (9)

### Angle between two planes, `angle_between_planes`

The acute (dihedral) angle between two planes given by their equations (only the normals matter).

$$\cos\theta = \frac{|\mathbf{n}_1 \cdot \mathbf{n}_2|}{\|\mathbf{n}_1\|\|\mathbf{n}_2\|}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a1` | in | x-coefficient of plane 1 | , |
| `b1` | in | y-coefficient of plane 1 | , |
| `c1` | in | z-coefficient of plane 1 | , |
| `a2` | in | x-coefficient of plane 2 | , |
| `b2` | in | y-coefficient of plane 2 | , |
| `c2` | in | z-coefficient of plane 2 | , |
| `angle` | out | angle between the planes, degrees, 0 to 90 | , |

**Applies when:** nonzero normals; the constants do not affect the angle.

```
$ geometry solve angle_between_planes a1=0 b1=0 c1=1 a2=1 b2=0 c2=1
angle = 45
  by: Angle between two planes  (angle between the normals, folded to [0, 90]: atan2(|n1 x
n2|, |n1.n2|))
  a1 = 0
  b1 = 0
  c1 = 1
  a2 = 1
  b2 = 0
  c2 = 1
  assumes: nonzero normals; the constants do not affect the angle
```

**What the worked example means:** This calculation determines the acute angle between two planes by comparing their normal vectors. The orientation of each plane is defined solely by its coefficients for x, y, and z, while any constant terms in the plane equations are ignored because they do not affect the angle. Computing the dot product of the normals and dividing by the product of their magnitudes gives the cosine of the angle. The result, 45, represents the smallest angle formed where the planes intersect, always falling within the range of zero to ninety degrees.

**Distance between two lines in space (skew or parallel), `skew_lines_distance`**

The shortest distance between the lines P1 + s d1 and P2 + t d2 in three dimensions.

$$d = \frac{|(P_2 - P_1) \cdot (\mathbf{d}_1 \times \mathbf{d}_2)|}{\|\mathbf{d}_1 \times \mathbf{d}_2\|}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `p1` | in | a point on line 1 | , |
| `d1` | in | direction of line 1 | , |
| `p2` | in | a point on line 2 | , |
| `d2` | in | direction of line 2 | , |
| `distance` | out | shortest distance | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `parallel` | out | yes / no (exact test) | , |

**Applies when:** nonzero directions.

```
$ geometry solve skew_lines_distance "p1=[0, 0, 0]" "d1=[1, 0, 0]" "p2=[0, 1, 1]" "d2=[0,
1, 0]"
distance = 1
parallel = no
  by: Distance between two lines in space (skew or parallel)  (|(P2 - P1).(d1 x d2)| / |d1
x d2|; parallel lines: |(P2 - P1) x d1| / |d1|)
  assumes: nonzero directions
```

**What the worked example means:** This calculation determines the minimum separation between two lines in three-dimensional space using a point on each line and their direction vectors. The result, 1, represents the length of the common perpendicular segment connecting them. If the cross product of the direction vectors is zero, the lines are parallel, indicated by no, and a different formula involving the vector from one point to the other crossed with the first direction is used.

### Distance from a point to a plane, `point_plane_distance`

The distance from a point to the plane a x + b y + c z + d = 0 and the foot of the perpendicular.

$$d = \frac{|ap_x + bp_y + cp_z + d|}{\sqrt{a^2 + b^2 + c^2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `pz` | in | z of the point | , |
| `a` | in | coefficient of x | , |
| `b` | in | coefficient of y | , |
| `c` | in | coefficient of z | , |
| `d` | in | constant term | , |
| `distance` | out | \|a px + b py + c pz + d\| / sqrt(a^2 + b^2 + c^2) | , |
| `fx` | out | x of the foot | , |
| `fy` | out | y of the foot | , |
| `fz` | out | z of the foot | , |

**Applies when:** a, b, c not all 0.

```
$ geometry solve point_plane_distance px=1 py=2 pz=3 a=2 b=-1 c=2 d=-3
distance = 1
fx = 0.3333333333
fy = 2.333333333
fz = 2.333333333
  by: Distance from a point to a plane  (signed value of the plane's equation divided by
the normal's length)
  px = 1
  py = 2
  pz = 3
  a = 2
  b = -1
  c = 2
  d = -3
  assumes: a, b, c not all 0
```

**What the worked example means:** This calculation determines the shortest spatial gap between a specific point and a defined plane, alongside the exact coordinates where a perpendicular line from that point meets the surface. The resulting distance value of 1 represents this minimal separation, while the foot coordinates 0.333333, 2.33333, and 2.33333 mark the precise location on the plane closest to the input point. This geometric relationship is essential for understanding proximity and alignment in three-dimensional space.

**Intersection of a line and a plane, `line_plane_intersection`**

The point where the line P0 + t v meets the plane a x + b y + c z + d = 0.

$$t = -\frac{\mathbf{n}\cdot P_0 + d}{\mathbf{n}\cdot\mathbf{v}}, \quad X = P_0 + t\,\mathbf{v}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `x0` | in | x of a point on the line | , |
| `y0` | in | y of the point | , |
| `z0` | in | z of the point | , |
| `dx` | in | x of the direction | , |
| `dy` | in | y of the direction | , |
| `dz` | in | z of the direction | , |
| `a` | in | coefficient of x | , |
| `b` | in | coefficient of y | , |
| `c` | in | coefficient of z | , |
| `d` | in | constant term | , |
| `x` | out | x of the intersection | , |
| `y` | out | y | , |
| `z` | out | z | , |
| `t` | out | line parameter at the intersection | , |

**Applies when:** direction not zero; a, b, c not all 0; line not parallel to the plane (exact test).

```
$ geometry solve line_plane_intersection x0=1 y0=0 z0=0 dx=1 dy=2 dz=3 a=1 b=1 c=1 d=-7
x = 2
y = 2
z = 3
t = 1
  by: Intersection of a line and a plane  (t = -(n.P0 + d)/(n.v) in exact rational
arithmetic)
  x0 = 1
  y0 = 0
  z0 = 0
  dx = 1
  dy = 2
  dz = 3
  a = 1
  b = 1
  c = 1
  d = -7
  assumes: direction not zero; a, b, c not all 0; line not parallel to the plane (exact
test)
```

In the worked example, x0 = 1, y0 = 0, z0 = 0, dx = 1, dy = 2, dz = 3, a = 1, b = 1, c = 1, d = -7 go through the equation above and give x = 2, y = 2, z = 3, t = 1. It applies only when direction not zero; a, b, c not all 0; line not parallel to the plane (exact test): outside that it says what is wrong instead of returning a number.

**Line in space: vector, parametric and symmetric equations, `line_in_space`**

The vector, parametric and symmetric equations of the line in space through two points, or through a point with a given direction, its unit direction and the point at a given parameter t.

$$\mathbf{r} = \mathbf{P} + t\,\mathbf{v}, \qquad \frac{x - x_0}{a} = \frac{y - y_0}{b} = \frac{z - z_0}{c}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `point` | in | a point P on the line: x y z | , |
| `second_point` | in | a second point Q on the line: x y z (optional) | , |
| `direction` | in | a direction vector: x y z (optional) | , |
| `t` | in | a parameter value t (the point P + t v) (optional) | , |
| `vector_form` | out | r = P + t v | , |
| `parametric` | out | x = x0 + a t, y = y0 + b t, z = z0 + c t | , |
| `symmetric` | out | (x - x0)/a = (y - y0)/b = (z - z0)/c (a coordinate whose direction component is 0 stays fixed) | , |
| `x0` | out | x of P | , |
| `y0` | out | y of P | , |
| `z0` | out | z of P | , |
| `dx` | out | x of the direction v | , |
| `dy` | out | y of the direction v | , |
| `dz` | out | z of the direction v | , |
| `ux` | out | x of the unit direction | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `uy` | out | y of the unit direction | , |
| `uz` | out | z of the unit direction | , |
| `xt` | out | x of the point at t | , |
| `yt` | out | y of the point at t | , |
| `zt` | out | z of the point at t | , |

**Applies when:** either a second point or a direction (not both); the two points differ / the direction is not zero; convention: through two points the direction is Q - P.

```
$ geometry solve line_in_space "second_point=[4, 6, 9]" "point=[1, 2, 3]"
vector_form = r = (1, 2, 3) + t(3, 4, 6)
parametric = x = 1 + 3t, y = 2 + 4t, z = 3 + 6t
symmetric = (x - 1)/3 = (y - 2)/4 = (z - 3)/6
x0 = 1
y0 = 2
z0 = 3
dx = 3
dy = 4
dz = 6
ux = 0.3841106398
uy = 0.5121475197
uz = 0.7682212796
xt = null
yt = null
zt = null
  by: Line in space: vector, parametric and symmetric equations  (v = Q - P (or the given
direction); r = P + t v; eliminating t gives the symmetric equations)
  assumes: either a second point or a direction (not both); the two points differ / the
direction is not zero; convention: through two points the direction is Q - P
```

In the worked example, second_point = [4, 6, 9], point = [1, 2, 3] go through the equation above and give vector_form = r = (1, 2, 3) + t(3, 4, 6), parametric = x = 1 + 3t, y = 2 + 4t, z = 3 + 6t, symmetric = (x - 1)/3 = (y - 2)/4 = (z - 3)/6, x0 = 1. It applies only when either a second point or a direction (not both); the two points differ / the direction is not zero; convention: through two points the direction is Q - P: outside that it says what is wrong instead of returning a number.

### Plane through three points, `plane_through_points`

The equation a x + b y + c z + d = 0 of the plane through three points, with (a, b, c) a unit normal.

$$\mathbf{n} = (B - A) \times (C - A), \quad \mathbf{n} \cdot \mathbf{x} - \mathbf{n} \cdot A = 0$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the three points: x y z for each | , |
| `a` | out | x-component of the unit normal | , |
| `b` | out | y-component | , |
| `c` | out | z-component | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `d` | out | constant term | , |

**Applies when:** exactly 3 points, not collinear; convention: (a, b, c) has length 1 and its first nonzero component is positive.

```
$ geometry solve plane_through_points "points=[1, 0, 0, 0, 1, 0, 0, 0, 1]"
a = 0.5773502692
b = 0.5773502692
c = 0.5773502692
d = -0.5773502692
  by: Plane through three points  (normal n = (B - A) x (C - A) (exact), normalised; d = -
n.A)
  assumes: exactly 3 points, not collinear; convention: (a, b, c) has length 1 and its
first nonzero component is positive
```

**What the worked example means:** This calculation determines the equation of the unique plane that passes through three specified non-collinear points. Computing the cross product of vectors formed from these points gives a normal vector to the surface. This vector is then normalized to ensure it has a length of one, following the convention that the first nonzero component is positive. The resulting coefficients 0.57735, 0.57735, and 0.57735 define the orientation of the plane in space, while the constant term -0.57735 positions the plane relative to the origin. Together, they form the standard linear equation for the geometric surface.

**Sphere through four points (circumsphere of a tetrahedron), `sphere_four_points`**

Centre and radius of the sphere through four non-coplanar points.

$$|X - P_i|^2 = R^2 \ (i = 1..4) \Rightarrow 2(P_i - P_1) \cdot X = |P_i|^2 - |P_1|^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the four points: x y z for each | , |
| `cx` | out | x of the centre | , |
| `cy` | out | y of the centre | , |
| `cz` | out | z of the centre | , |
| `radius` | out | radius | , |

**Applies when:** exactly 4 points, not coplanar.

```
$ geometry solve sphere_four_points "points=[1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0]"
cx = 0.5
cy = 0.5
cz = 0.5
radius = 0.8660254038
  by: Sphere through four points (circumsphere of a tetrahedron)  (subtract the first
sphere equation from the others: a 3x3 linear system, solved exactly (Cramer))
  assumes: exactly 4 points, not coplanar
```

**What the worked example means:** This calculation determines the unique sphere that passes through all four provided points in three-dimensional space. By treating these points as vertices of a tetrahedron, the

method solves a system of linear equations to find the exact coordinates of the sphere's center, given as 0.5, 0.5, and 0.5. The distance from this center to any of the original points defines the radius, which is calculated as 0.866025. This result represents the smallest sphere that can completely enclose the tetrahedron formed by the inputs.

**Tetrahedron from four vertices: volume and surface area, `tetrahedron_measures`**

Volume and total surface area of the tetrahedron with four given vertices.

$$V = \tfrac{1}{6}\left|(B-A)\cdot\big((C-A)\times(D-A)\big)\right|$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the four vertices: x y z for each | , |
| `volume` | out | \|det(B - A, C - A, D - A)\| / 6 | , |
| `surface_area` | out | sum of the four face areas | , |

**Applies when:** exactly 4 points; not coplanar.

```
$ geometry solve tetrahedron_measures "points=[0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1]"
volume = 0.1666666667
surface_area = 2.366025404
  by: Tetrahedron from four vertices: volume and surface area  (scalar triple product; face
areas from cross products)
  assumes: exactly 4 points; not coplanar
```

**What the worked example means:** This calculation determines the geometric properties of a tetrahedron defined by four specific points in space. Using the scalar triple product, it computes the enclosed volume as 0.166667, which represents the amount of three-dimensional space occupied by the shape. It also calculates the total surface area as 2.36603, which is the sum of the areas of all four triangular faces formed by these vertices.

**Triangle in space: area and perimeter from vertices, `triangle_area_3d`**

Area and perimeter of the triangle with vertices A, B, C in three-dimensional space.

$$K = \tfrac{1}{4}\sqrt{(a+b+c)(-a+b+c)(a-b+c)(a+b-c)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the vertices: xA yA zA, xB yB zB, xC yC zC | , |
| `area` | out | area | , |
| `perimeter` | out | perimeter | , |

**Applies when:** exactly 3 points, not collinear.

```
$ geometry solve triangle_area_3d "points=[1, 0, 0, 0, 2, 0, 0, 0, 3]"
area = 3.5
perimeter = 9.003896913
  by: Triangle in space: area and perimeter from vertices  (side lengths, then Heron's
formula in Kahan's stable form)
  assumes: exactly 3 points, not collinear
```

**What the worked example means:** This calculation determines the size of the triangle formed by the three points provided in space. It first computes the length of each side using the distance formula, then sums these lengths to find the perimeter. The area is derived from these side lengths using Heron's formula, which relates the sides to the enclosed surface. For the input points [1, 0, 0, 0, 2, 0, 0, 0, 3], the resulting area is 3.5 and the total boundary length is 9.0039.

## Solid geometry (2)

### Solid angle of a cone, `solid_angle_cone`

The solid angle (steradians) subtended at the apex by a right circular cone of given half-angle.

$$\Omega = 2\pi(1 - \cos\alpha) = 4\pi\sin^2\tfrac{\alpha}{2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `half_angle` | in | half-angle of the cone (axis to side), degrees | , |
| `solid_angle` | out | 2 pi (1 - cos a) steradians | , |
| `fraction_of_sphere` | out | solid angle / (4 pi) | , |

**Applies when:** 0 < half-angle <= 180 degrees; an apex (full opening) angle in the question is halved.

```
$ geometry solve solid_angle_cone half_angle=60
solid_angle = 3.141592654
fraction_of_sphere = 0.25
  by: Solid angle of a cone  (Omega = 2 pi (1 - cos a) = 4 pi sin^2(a/2) (no cancellation
for small a))
  half_angle = 60
  assumes: 0 < half-angle <= 180 degrees; an apex (full opening) angle in the question is
halved
```

**What the worked example means:** This calculation determines the solid angle subtended at the apex by a right circular cone with a half-angle of 60. The result, 3.14159 steradians, quantifies the portion of three-dimensional space enclosed by the cone. Expressed as a fraction of the full spherical surface, this value is 0.25, indicating how much of the surrounding environment the cone covers from the perspective of its tip.

### Solid angle subtended by a triangle, `solid_angle_triangle`

The solid angle (steradians) that the triangle with vertices A, B, C subtends at the origin.

$$\tan\frac{\Omega}{2} = \frac{|\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c})|}{abc + (\mathbf{a}\cdot\mathbf{b})c + (\mathbf{a}\cdot\mathbf{c})b + (\mathbf{b}\cdot\mathbf{c})a}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the vertices as seen from the origin: x y z for each | , |
| `solid_angle` | out | solid angle, steradians | , |

**Applies when:** no vertex at the origin; the triangle's plane does not pass through the origin; observer at the origin.

```
$ geometry solve solid_angle_triangle "points=[1, 0, 0, 0, 1, 0, 0, 0, 1]"
solid_angle = 1.570796327
  by: Solid angle subtended by a triangle  (Van Oosterom-Strackee: tan(Omega/2) = |a.(b x
c)| / (abc + (a.b)c + (a.c)b + (b.c)a))
  assumes: no vertex at the origin; the triangle's plane does not pass through the origin;
observer at the origin
```

**What the worked example means:** This calculation determines the angular size of a triangle in three-dimensional space as viewed from the origin. By using the vertices provided, it computes the steradians covered by the shape. The resulting value, 1.5708, represents how much of the observer's field of view is occupied by the triangle.

## Solids (10)

### Cone: volume, slant height and surface area, `cone_measures`

Volume, slant height, lateral area and total surface area of a right circular cone.

$$V = \tfrac{1}{3}\pi r^2 h, \quad l = \sqrt{r^2 + h^2}, \quad A_L = \pi r l$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius of the base | , |
| `height` | in | perpendicular height | , |
| `volume` | out | pi r^2 h / 3 | , |
| `slant_height` | out | sqrt(r^2 + h^2) | , |
| `lateral_area` | out | pi r l | , |
| `total_area` | out | pi r (r + l) | , |

**Applies when:** radius, height > 0.

```
$ geometry solve cone_measures radius=3 height=4
volume = 37.69911184
slant_height = 5
lateral_area = 47.1238898
total_area = 75.39822369
  by: Cone: volume, slant height and surface area  (V = pi r^2 h/3; l = sqrt(r^2 + h^2);
lateral = pi r l)
  radius = 3
  height = 4
  assumes: radius, height > 0
```

**What the worked example means:** This calculation determines the geometric properties of a right circular cone based on its base radius of 3 and perpendicular height of 4. The volume is found to be 37.6991, representing the space enclosed by the shape. The slant height, which is the distance from the apex to the edge of the base, is calculated as 5. Using these values, the lateral surface area is 47.1239, and the total surface area including the base is 75.3982. These metrics are essential for understanding the physical dimensions and material requirements of conical objects.

**Cuboid (rectangular box): volume, surface area, space diagonal, `cuboid_measures`**

Volume, total surface area and space diagonal of a rectangular box (a cube when all edges are equal).

$$V = lwh, \quad S = 2(lw + lh + wh), \quad d = \sqrt{l^2 + w^2 + h^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `length` | in | length | , |
| `width` | in | width | , |
| `height` | in | height | , |
| `volume` | out | l w h | , |
| `surface_area` | out | 2(lw + lh + wh) | , |
| `space_diagonal` | out | sqrt(l^2 + w^2 + h^2) | , |

**Applies when:** length, width, height > 0.

```
$ geometry solve cuboid_measures length=3 width=4 height=12
volume = 144
surface_area = 192
space_diagonal = 13
  by: Cuboid (rectangular box): volume, surface area, space diagonal  (V = l w h; S = 2(lw
+ lh + wh); d = sqrt(l^2 + w^2 + h^2))
  length = 3
  width = 4
  height = 12
  assumes: length, width, height > 0
```

**What the worked example means:** This calculation determines the volume, total surface area, and space diagonal of a rectangular box using the given length, width, and height. The volume represents the amount of three-dimensional space inside the box, while the surface area indicates the total area of all six faces required to cover it. The space diagonal measures the straight-line distance between two opposite corners passing through the interior. With a calculated volume of 144 and surface area of 192, these values help quantify the physical properties of the object based on its dimensions.

**Cylinder: volume and surface area, `cylinder_measures`**

Volume, lateral (curved) area and total surface area of a right circular cylinder.

$$V = \pi r^2 h, \quad A = 2\pi r h + 2\pi r^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius of the base | , |
| `height` | in | height | , |
| `volume` | out | pi r^2 h | , |
| `lateral_area` | out | 2 pi r h | , |
| `total_area` | out | 2 pi r (r + h) | , |

**Applies when:** radius, height > 0.

```
$ geometry solve cylinder_measures radius=3 height=5
volume = 141.3716694
lateral_area = 94.24777961
total_area = 150.7964474
  by: Cylinder: volume and surface area  (V = pi r^2 h; lateral = 2 pi r h; total = lateral
+ 2 pi r^2)
  radius = 3
  height = 5
  assumes: radius, height > 0
```

**What the worked example means:** This calculation determines the capacity and surface extent of a right circular cylinder using its radius and height. The volume, which represents the space inside the cylinder, is calculated as pi times the square of the radius times the height. The lateral area, representing the curved side surface, is found by multiplying two pi times the radius times the height. The total surface area includes both the lateral area and the areas of the two circular bases. For this specific geometry, the resulting volume is 141.372, the lateral area is 94.2478, and the total surface area is 150.796. These values are essential for understanding material requirements and storage capacity in practical applications.

**Frustum of a cone: volume and surface area, `frustum_measures`**

Volume, slant height, lateral and total surface area of a frustum of a right circular cone.

$$V = \tfrac{1}{3}\pi h(R^2 + Rr + r^2), \quad A_L = \pi(R + r)\sqrt{h^2 + (R - r)^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `r1` | in | radius of one base | , |
| `r2` | in | radius of the other base | , |
| `height` | in | perpendicular height | , |
| `volume` | out | pi h (R^2 + R r + r^2)/3 | , |
| `slant_height` | out | sqrt(h^2 + (R - r)^2) | , |
| `lateral_area` | out | pi (R + r) s | , |
| `total_area` | out | lateral + pi (R^2 + r^2) | , |

**Applies when:** r1, r2, height > 0.

```
$ geometry solve frustum_measures height=4 r1=3 r2=6
volume = 263.8937829
slant_height = 5
lateral_area = 141.3716694
total_area = 282.7433388
  by: Frustum of a cone: volume and surface area  (V = pi h (R^2 + R r + r^2)/3; s =
sqrt(h^2 + (R - r)^2); lateral = pi (R + r) s)
  height = 4
  r1 = 3
  r2 = 6
  assumes: r1, r2, height > 0
```

**What the worked example means:** This calculation determines the capacity and exterior surface coverage of a right circular cone frustum. Using the base radii 3 and 6 along with the perpendicular height 4, the model computes the slant height as 5. The resulting volume is 263.894, while the lateral surface area is 141.372. Adding the areas of both circular bases to the lateral area yields a total surface area of 282.743, which represents the complete material required to enclose the shape.

**Right prism on a regular polygon base: volume and surface area, `regular_prism`**

Volume, base area, lateral and total surface area of a right prism whose base is a regular n-gon.

$$B = \frac{n s^2}{4 \tan(\pi/n)}, \quad V = Bh, \quad A = nsh + 2B$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `n` | in | number of sides of the base | , |
| `side` | in | side length of the base | , |
| `height` | in | height (length) of the prism | , |
| `volume` | out | base area * height | , |
| `lateral_area` | out | n s h | , |
| `total_area` | out | lateral + 2 * base | , |
| `base_area` | out | n s^2 / (4 tan(pi/n)) | , |

**Applies when:** 3 <= n <= 200; side, height > 0; a triangular prism is read only when 'regular' or 'equilateral' is said.

```
$ geometry solve regular_prism n=6 side=2 height=10
volume = 103.9230485
lateral_area = 120
total_area = 140.7846097
base_area = 10.39230485
  by: Right prism on a regular polygon base: volume and surface area  (base area n s^2/(4
tan(pi/n)); V = B h; lateral = perimeter * h)
  side = 2
  height = 10
  assumes: 3 <= n <= 200; side, height > 0; a triangular prism is read only when 'regular'
or 'equilateral' is said
```

**What the worked example means:** This calculation determines the geometric properties of a right prism whose base is a regular polygon. Using the number of sides 6, the side length 2, and the prism's height 10 shows the base area to be 10.3923. The volume, representing the space enclosed by the solid, is calculated as 103.923. The lateral area, which is the sum of the rectangular side faces, equals 120, while the total surface area, including the two bases, is 140.785. These values describe the physical dimensions and capacity of the prismatic shape.

**Right rectangular (or square) pyramid: volume and surface area, `pyramid_measures`**

Volume, lateral area and total surface area of a right pyramid on a rectangular base (apex above the base centre).

$$V = \tfrac{1}{3}lwh, \quad A_L = l\sqrt{h^2 + (w/2)^2} + w\sqrt{h^2 + (l/2)^2}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `length` | in | base length | , |
| `width` | in | base width | , |
| `height` | in | perpendicular height | , |
| `volume` | out | l w h / 3 | , |
| `lateral_area` | out | area of the four triangular faces | , |
| `total_area` | out | lateral + base | , |

**Applies when:** length, width, height > 0; the apex is directly above the centre of the base.

```
$ geometry solve pyramid_measures height=4 length=6 width=6
volume = 48
lateral_area = 60
total_area = 96
  by: Right rectangular (or square) pyramid: volume and surface area  (V = lwh/3; each pair
of faces has slant height sqrt(h^2 + (other side/2)^2))
  height = 4
  length = 6
  width = 6
  assumes: length, width, height > 0; the apex is directly above the centre of the base
```

**What the worked example means:** This calculation determines the space enclosed by a right pyramid with a rectangular base, using a length of 6, width of 6, and height of 4. The resulting volume of 48 represents the capacity within the shape. Additionally, the lateral area of 60 covers the four triangular sides, while the total surface area of 96 includes the base as well.

**Sphere: volume and surface area, `sphere_measures`**

Volume and surface area of a sphere from its radius (or diameter).

$$V = \tfrac{4}{3}\pi r^3, \quad S = 4\pi r^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius | , |
| `volume` | out | 4/3 pi r^3 | , |
| `surface_area` | out | 4 pi r^2 | , |

**Applies when:** radius > 0.

```
$ geometry solve sphere_measures radius=3
volume = 113.0973355
surface_area = 113.0973355
  by: Sphere: volume and surface area  (V = 4/3 pi r^3; S = 4 pi r^2)
  radius = 3
  assumes: radius > 0
```

**What the worked example means:** This calculation determines the three-dimensional space enclosed by a sphere and the total area of its outer surface. Given a positive radius of 3, the volume is computed as four thirds pi times the cube of the radius, resulting in 113.097. The surface area is calculated as four pi times the square of the radius, yielding 113.097. These values describe how much material fills the object and how much covering is needed to wrap it.

### Spherical cap: volume, curved area, base radius, `spherical_cap`

Volume, curved surface area and base radius of a cap of height h cut from a sphere of radius R.

$$V = \tfrac{1}{3}\pi h^2(3R - h), \quad A = 2\pi R h, \quad a = \sqrt{h(2R - h)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `radius` | in | radius R of the sphere | , |
| `height` | in | height h of the cap | , |
| `volume` | out | pi h^2 (3R - h)/3 | , |
| `cap_area` | out | 2 pi R h | , |
| `base_radius` | out | sqrt(h (2R - h)) | , |

**Applies when:** R > 0; 0 < h <= 2R.

```
$ geometry solve spherical_cap radius=5 height=2
volume = 54.45427266
cap_area = 62.83185307
base_radius = 4
  by: Spherical cap: volume, curved area, base radius  (V = pi h^2 (3R - h)/3; curved area
2 pi R h (Archimedes))
  radius = 5
  height = 2
  assumes: R > 0; 0 < h <= 2R
```

**What the worked example means:** This calculation determines the geometric properties of a spherical cap defined by a sphere radius of 5 and a cap height of 2. The resulting volume of 54.4543 represents the space enclosed within the cap, while the curved surface area of 62.8319 quantifies the exterior skin size.

Additionally, the base radius of 4 indicates the width of the circular boundary where the cap meets the cutting plane.

**Spheroid (ellipsoid of revolution): surface area and volume, `spheroid_measures`**

Surface area and volume of an oblate or prolate spheroid with equatorial radius a and polar radius c.

$$S_{\text{obl}} = 2\pi a^2\left(1 + \frac{1-e^2}{e}\operatorname{artanh} e\right), \quad S_{\text{pro}} = 2\pi a^2\left(1 + \frac{c}{ae}\arcsin e\right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `a` | in | equatorial radius (semi-axis in the equator) | , |
| `c` | in | polar radius (semi-axis along the axis) | , |
| `surface_area` | out | surface area | , |
| `volume` | out | 4/3 pi a^2 c | , |
| `shape` | out | oblate / prolate / sphere | , |

**Applies when:** a, c > 0.

```
$ geometry solve spheroid_measures a=1 c=2
surface_area = 21.47843533
volume = 8.37758041
shape = prolate
  by: Spheroid (ellipsoid of revolution): surface area and volume  (oblate: S = 2 pi a^2 (1
+ (1 - e^2)/e atanh e); prolate: S = 2 pi a^2 (1 + c/(a e) asin e); e the eccentricity)
  a = 1
  c = 2
  assumes: a, c > 0
```

**What the worked example means:** This calculation determines the surface area and volume for a spheroid defined by an equatorial radius of 1 and a polar radius of 2. The resulting surface area is 21.4784 and the volume is 8.37758. Based on the relationship between these radii, the shape is identified as prolate, indicating whether it is flattened or elongated compared to a sphere.

**Torus: volume and surface area, `torus_measures`**

Volume and surface area of a ring torus with major radius R (centre to tube centre) and tube radius r.

$$V = 2\pi^2 R r^2, \quad A = 4\pi^2 R r$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `major` | in | major radius R | , |
| `minor` | in | tube (minor) radius r | , |
| `volume` | out | 2 pi^2 R r^2 | , |
| `surface_area` | out | 4 pi^2 R r | , |

**Applies when:** 0 < r < R.

```
$ geometry solve torus_measures major=5 minor=2
volume = 394.784176
surface_area = 394.784176
  by: Torus: volume and surface area  (Pappus's theorems: area/volume of the revolved
circle times the path 2 pi R)
  major = 5
  minor = 2
  assumes: 0 < r < R
```

**What the worked example means:** This calculation determines the volume and surface area of a ring torus using Pappus's centroid theorems. It takes the major radius, which is the distance from the center of the tube to the center of the torus, and the minor radius, which is the radius of the circular cross-section. The resulting volume is 394.784, representing the space enclosed by the shape, while the surface area is 394.784, indicating the total extent of its outer skin.

## Spherical geometry (3)

### Area and spherical excess of a spherical triangle, `spherical_triangle_area`

The spherical excess and area of the triangle with three vertices given by latitude and longitude on a sphere of radius R.

$$\tan\frac{E}{4} = \sqrt{\tan\frac{s}{2}\tan\frac{s-a}{2}\tan\frac{s-b}{2}\tan\frac{s-c}{2}}, \quad A = ER^2$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `points` | in | the vertices (lat lon): lat1 lon1, lat2 lon2, lat3 lon3, degrees | , |
| `radius` | in | sphere radius (default 1; 'Earth' means 6371 km) (optional) | , |
| `excess` | out | spherical excess A + B + C - 180, degrees | , |
| `area` | out | area E R^2 (E in radians) | , |

**Applies when:** the smaller triangle (each side less than 180 degrees); vertices not on one great circle; radius defaults to 1.

```
$ geometry solve spherical_triangle_area radius=1 "points=[0, 0, 0, 90, 90, 0]"
excess = 90
area = 1.570796327
  by: Area and spherical excess of a spherical triangle  (side arcs by the haversine
formula, then L'Huilier's theorem)
  radius = 1
  assumes: the smaller triangle (each side less than 180 degrees); vertices not on one
great circle; radius defaults to 1
```

**What the worked example means:** This calculation determines the area of a triangle on a sphere using three vertices defined by latitude and longitude. It first computes the side lengths in radians, then applies L'Huilier's theorem to find the spherical excess, which is reported as 90. Finally, it multiplies this excess by the square of the radius to obtain the surface area, shown as 1.5708. The result represents the actual geometric size of the region bounded by the great circle arcs connecting the points.

**Great-circle distance between two points on a sphere, `great_circle_distance`**

The central angle and great-circle distance between two points given by latitude and longitude (on the Earth unless a radius is given).

$$h = \sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1\cos\varphi_2\sin^2\frac{\Delta\lambda}{2}, \quad d = 2R\,\operatorname{atan2}(\sqrt{h}, \sqrt{1-h})$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `lat1` | in | latitude of point 1, degrees | , |
| `lon1` | in | longitude of point 1, degrees | , |
| `lat2` | in | latitude of point 2 | , |
| `lon2` | in | longitude of point 2 | , |
| `radius` | in | sphere radius (default 6371, the mean Earth radius in km) (optional) | , |
| `central_angle` | out | central angle, degrees | , |
| `distance` | out | great-circle distance (radius units) | , |

**Applies when:** points written (latitude, longitude) in signed decimal degrees; radius defaults to 6371 km (mean Earth radius; the Earth is treated as a sphere).

```
$ geometry solve great_circle_distance radius=1 lat1=0 lon1=0 lat2=0 lon2=90
central_angle = 90
distance = 1.570796327
  by: Great-circle distance between two points on a sphere  (haversine formula in its atan2
form)
  radius = 1
  lat1 = 0
  lon1 = 0
  lat2 = 0
  lon2 = 90
  assumes: points written (latitude, longitude) in signed decimal degrees; radius defaults
to 6371 km (mean Earth radius; the Earth is treated as a sphere)
```

**What the worked example means:** This calculation determines the shortest path along the surface of a sphere, known as the great-circle distance, between two locations defined by their latitude and longitude. Using the haversine formula adapted for numerical stability via the atan2 function, it computes the central angle separating the two points and multiplies this angle by the sphere's radius to yield the linear distance. The result, 1.5708, represents the physical separation in the units of the provided radius, while 90 indicates the angular span on the globe.

**Initial bearing (forward azimuth) along a great circle, `initial_bearing`**

The initial compass bearing from point 1 to point 2 along the great circle, degrees clockwise from north.

$$\theta = \operatorname{atan2}\left(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda\right)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `lat1` | in | latitude of the start, degrees | , |

| symbol | | meaning | SI unit |
|---|---|---|---|
| `lon1` | in | longitude of the start | , |
| `lat2` | in | latitude of the destination | , |
| `lon2` | in | longitude of the destination | , |
| `bearing` | out | initial bearing, degrees in [0, 360), clockwise from north | , |

**Applies when:** (latitude, longitude) in signed decimal degrees; the start is not a pole; the points are neither equal nor antipodal.

```
$ geometry solve initial_bearing lat1=0 lon1=0 lat2=0 lon2=90
bearing = 90
  by: Initial bearing (forward azimuth) along a great circle  (theta = atan2(sin dl cos f2,
cos f1 sin f2 - sin f1 cos f2 cos dl))
  lat1 = 0
  lon1 = 0
  lat2 = 0
  lon2 = 90
  assumes: (latitude, longitude) in signed decimal degrees; the start is not a pole; the
points are neither equal nor antipodal
```

**What the worked example means:** This calculation determines the starting compass direction required to follow the shortest path, or great circle, between two geographic points. Using the latitude and longitude of the start point, which are 0 and 0, along with those of the destination at 0 and 90, the method computes the angle relative to true north. The resulting value of 90 represents the initial heading in degrees within a range of zero to three hundred sixty, essential for accurate navigation planning.

## Surfaces (2)

### Curvature of a torus at a point, `torus_curvature`

Gaussian, mean and principal curvatures of a ring torus at tube angle v (v = 0 on the outer equator, 180 on the inner).

$$K = \frac{\cos v}{r(R + r\cos v)}, \quad H = \frac{R + 2r\cos v}{2r(R + r\cos v)}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `major` | in | major radius R | , |
| `minor` | in | tube radius r | , |
| `v` | in | tube angle v, degrees (0 = outer equator) | , |
| `gaussian` | out | K = cos v / (r (R + r cos v)) | , |
| `mean` | out | H = (R + 2 r cos v) / (2 r (R + r cos v)) | , |
| `k1` | out | principal curvature around the tube, 1/r | , |
| `k2` | out | principal curvature along the ring, cos v / (R + r cos v) | , |

**Applies when:** 0 < r < R; signs: curvatures positive where the surface bends away from the outward normal (a sphere is positive).

```
$ geometry solve torus_curvature major=3 minor=1 v=0
gaussian = 0.25
mean = 0.625
k1 = 1
k2 = 0.25
  by: Curvature of a torus at a point  (principal curvatures of the surface of revolution:
1/r around the tube and cos v/(R + r cos v) along the ring)
  major = 3
  minor = 1
  v = 0
  assumes: 0 < r < R; signs: curvatures positive where the surface bends away from the
outward normal (a sphere is positive)
```

**What the worked example means:** This calculation determines the curvature properties of a ring torus at a specific tube angle. Given a major radius of 3 and a minor radius of 1, the surface bends with a principal curvature around the tube of 1 and along the ring of 0.25. These combine to yield a Gaussian curvature of 0.25 and a mean curvature of 0.625. Positive values indicate bending away from the outward normal, consistent with standard sphere conventions.

**Gaussian and mean curvature of a quadratic surface z = f(x, y), `surface_curvature`**

Gaussian curvature K and mean curvature H of z = A x^2 + B xy + C y^2 + D x + E y + F at the point above (x0, y0).

$$K = \frac{f_{xx} f_{yy} - f_{xy}^2}{(1 + f_x^2 + f_y^2)^2}, \quad H = \frac{(1 + f_y^2) f_{xx} - 2 f_x f_y f_{xy} + (1 + f_x^2) f_{yy}}{2(1 + f_x^2 + f_y^2)^{3/2}}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `A` | in | coefficient of x^2 | , |
| `B` | in | coefficient of xy | , |
| `C` | in | coefficient of y^2 | , |
| `D` | in | coefficient of x | , |
| `E` | in | coefficient of y | , |
| `x0` | in | x of the point | , |
| `y0` | in | y of the point | , |
| `F` | in | constant term (does not affect curvature) (optional) | , |
| `gaussian` | out | Gaussian curvature K | , |
| `mean` | out | mean curvature H (upward normal: positive where the surface opens upward) | , |

**Applies when:** the unit normal points upward, (-f_x, -f_y, 1)/|...|.

```
$ geometry solve surface_curvature A=1 B=0 C=1 D=0 E=0 F=0 x0=0 y0=0
gaussian = 4
mean = 2
  by: Gaussian and mean curvature of a quadratic surface z = f(x, y)  (Monge patch: K =
(f_xx f_yy - f_xy^2)/(1 + f_x^2 + f_y^2)^2, H = ((1 + f_y^2) f_xx - 2 f_x f_y f_xy + (1 +
f_x^2) f_yy) / (2 (1 + f_x^2 + f_y^2)^(3/2)))
  A = 1
  B = 0
  C = 1
  D = 0
  E = 0
  F = 0
  x0 = 0
  y0 = 0
  assumes: the unit normal points upward, (-f_x, -f_y, 1)/|...|
```

**What the worked example means:** This calculation determines the Gaussian and mean curvatures for a quadratic surface defined by coefficients A, B, C, D, E, and F at the specific point x0, y0. The Gaussian curvature, reported as 4, measures the intrinsic bending of the surface, indicating whether the local shape is elliptic or hyperbolic. The mean curvature, given as 2, represents the average bending relative to the upward-pointing normal vector, where positive values indicate the surface opens upward.

## Transformations (4)

### Dilation (enlargement) of a point, `dilate_point`

The image of a point under a dilation with scale factor k about a centre (the origin unless given).

$$P' = C + k\,(P - C)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `k` | in | scale factor (negative: through the centre) | , |
| `cx` | in | x of the centre (default 0) (optional) | , |
| `cy` | in | y of the centre (default 0) (optional) | , |
| `x` | out | x of the image | , |
| `y` | out | y of the image | , |

**Applies when:** k != 0; centre defaults to the origin.

```
$ geometry solve dilate_point k=3 px=2 py=3
x = 6
y = 9
  by: Dilation (enlargement) of a point  (P' = C + k (P - C))
  k = 3
  px = 2
  py = 3
  assumes: k != 0; centre defaults to the origin
```

**What the worked example means:** This calculation determines the new position of a point after applying a dilation with scale factor 3 relative to a specified center. Taking the original point's coordinates, subtracting the center's coordinates to find the vector from the center, multiplying that vector by the scale factor, and adding the center back gives the image point. The resulting x-coordinate is 6 and the y-coordinate is 9. This process ensures the distance from the center scales uniformly according to the factor provided.

### Reflection of a point in a line, `reflect_point_line`

The mirror image of a point in the line a x + b y + c = 0 (including the axes and y = x).

$$P' = P - 2\,\frac{a p_x + b p_y + c}{a^2 + b^2}\,(a, b)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `a` | in | coefficient of x of the line | , |
| `b` | in | coefficient of y of the line | , |
| `c` | in | constant of the line | , |
| `x` | out | x of the image | , |
| `y` | out | y of the image | , |

**Applies when:** a and b not both 0.

```
$ geometry solve reflect_point_line px=3 py=4 a=-1 b=1 c=0
x = 4
y = 3
  by: Reflection of a point in a line  (P' = P - 2 (a px + b py + c)/(a^2 + b^2) (a, b))
  px = 3
  py = 4
  a = -1
  b = 1
  c = 0
  assumes: a and b not both 0
```

**What the worked example means:** This calculation determines the mirror image of a specific point across a given straight line in the Cartesian plane. Using the coordinates of the original point, provided as 3 for the horizontal position and 4 for the vertical position, along with the line parameters -1, 1, and 0, the algorithm computes the new location. The resulting reflected point has an x-coordinate of 4 and a y-coordinate of 3. This transformation preserves distance from the mirror line but reverses the direction perpendicular to it, effectively creating a symmetric counterpart.

### Rotation of a point, `rotate_point`

The image of a point rotated by an angle about a centre (the origin unless given); counterclockwise is positive.

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix} + \begin{pmatrix} c_x \\ c_y \end{pmatrix}$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `px` | in | x of the point | , |
| `py` | in | y of the point | , |
| `angle` | in | rotation angle in degrees, counterclockwise positive | , |
| `cx` | in | x of the centre (default 0) (optional) | , |
| `cy` | in | y of the centre (default 0) (optional) | , |
| `x` | out | x of the image | , |
| `y` | out | y of the image | , |

**Applies when:** angle in degrees, counterclockwise positive ('clockwise' in a question makes it negative); centre defaults to the origin.

```
$ geometry solve rotate_point angle=90 px=3 py=4
x = -4
y = 3
  by: Rotation of a point  (x' = cx + (x - cx) cos t - (y - cy) sin t, y' = cy + (x - cx)
sin t + (y - cy) cos t (exact for multiples of 90 degrees))
  angle = 90
  px = 3
  py = 4
  assumes: angle in degrees, counterclockwise positive ('clockwise' in a question makes it
negative); centre defaults to the origin
```

In the worked example, angle = 90, px = 3, py = 4 go through the equation above and give x = -4, y = 3. It applies only when angle in degrees, counterclockwise positive ('clockwise' in a question makes it negative); centre defaults to the origin: outside that it says what is wrong instead of returning a number.

### Rotation of a point in space about an axis, `rotate_point_3d`

The image of a point rotated by an angle about an axis through the origin (right-hand rule).

$$\mathbf{v}' = \mathbf{v}\cos\theta + (\mathbf{k}\times\mathbf{v})\sin\theta + \mathbf{k}(\mathbf{k}\cdot\mathbf{v})(1-\cos\theta)$$

| symbol | | meaning | SI unit |
|---|---|---|---|
| `point` | in | the point x y z | , |
| `axis` | in | axis direction (through the origin) | , |
| `angle` | in | angle in degrees (right-hand rule) | , |
| `x` | out | x of the image | , |
| `y` | out | y of the image | , |
| `z` | out | z of the image | , |

**Applies when:** axis not zero; it passes through the origin; positive angle: counterclockwise looking from the axis tip toward the origin.

```
$ geometry solve rotate_point_3d angle=90 "axis=[0, 0, 1]" "point=[1, 0, 0]"
x = 0
y = 1
z = 0
  by: Rotation of a point in space about an axis  (Rodrigues' formula v' = v cos t + (k x
v) sin t + k (k.v)(1 - cos t), exact for multiples of 90 degrees)
  angle = 90
  assumes: axis not zero; it passes through the origin; positive angle: counterclockwise
looking from the axis tip toward the origin
```

In the worked example, angle = 90, axis = [0, 0, 1], point = [1, 0, 0] go through the equation above and give x = 0, y = 1, z = 0. It applies only when axis not zero; it passes through the origin; positive angle: counterclockwise looking from the axis tip toward the origin: outside that it says what is wrong instead of returning a number.

Questions and reports: support@ai999x.com. AI999X, The Future Is Self-Improving.